

# DisDP: Disaggregating Compute, Network, and Storage for Model-Sharded Data-Parallel Training

Mo Sun, Zihan Yang, Changyue Liao, Yingtao Li, Jie Zhang, Kaiqi Chen, Fei Wu and Zeke Wang  
Zhejiang University, China

{sunmo,zihanyang,changyueliao,Li\_Yingtao,carlzhang4,chiaki\_cage,wufei,wangzeke}@zju.edu.cn

**Abstract**—Model-sharded data parallelism (MSDP), e.g., ZeRO, evenly shards the model states across all GPUs, and thus has been widely adopted by LLM pre-training, such as Llama and DeepSeek, due to its low GPU memory capacity requirement. However, MSDP introduces severe overhead from additional network communication collectives (i.e., AllGather and ReduceScatter). Although the collectives themselves only occupy fewer than 10% of GPU SMs, their execution time increases by 41% due to the serial execution of aggregated CPU/GPU-managed compute (i.e., GEMM), network (i.e., NCCL), and storage (i.e., optimizer states). To this end, we present DisDP, a fully disaggregated distributed data-parallel architecture that first fully disaggregates compute, network, and storage for MSDP, such that GPUs only focus on the computing part, and thus the GPU utilization is maximized. The key idea is 1) fully offloading collectives to SmartNICs and SmartSwitch to avoid interference between GEMM kernels and collective kernels, and 2) fully offloading storage to a SmartSwitch-enhanced parameter server that allows a single PS to serve massive workers with linear scalability. DisDP on 8 distributed GPUs outperforms the state-of-the-art training systems by  $3.98\times$  when training on a 175B model, validating the efficiency of disaggregation.

## I. INTRODUCTION

Large language models (LLMs) have made advances in application domains such as natural language processing [1], [2], programming [3], and computer vision [4]–[6]. Along with the advances of LLM are their fast-growing model sizes, from 100M-scale [1] to 100B-scale [7]–[10]. Training a large-scale model requires using many GPUs with efficient parallelism strategies [11]–[29]. Among these strategies, model replicated data parallelism (MRDP or DP) [30]–[48] replicates the model states across GPU workers, and each GPU needs to accommodate the entire model state, including all parameters and auxiliary optimizer states. So, an NVIDIA H100 GPU cluster (each GPU with 80 GB of device memory) even fails to train a 7B model (with 112 GB gradients and model states), regardless of the number of GPUs in the cluster.

**Model-Sharded DP (MSDP).** To address the limitation of trainable model size, memory-efficient MSDP such as ZeRO [49]–[62] and PyTorch FSDP [51] are proposed to evenly shard the model states across all GPU workers, as shown in Figure 1(a).<sup>1</sup> With enough GPUs, MSDP allows training a huge model, and thus has been adopted by DeepSeek [63] and Llama [64] to pre-train 100B-scale mod-

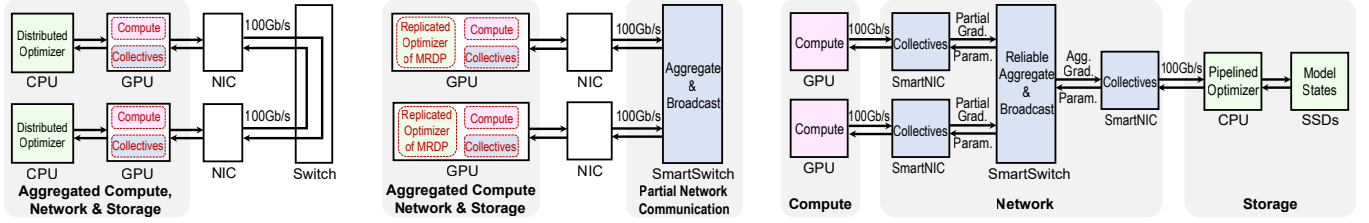
els.<sup>2</sup> Therefore, the optimization of MSDP itself is important. However, MSDP comes at the cost of extra collective operations, i.e., AllGather, to fetch on-demand parameters within an iteration. We observe that although network communication itself consumes relatively low GPU resources, its aggregated computation and collectives on GPUs lead to severe interference. In particular, with GPUs’ non-preemptive scheduling policy, current-round compute-intensive GEMM kernels would occupy all GPU SMs and block next-round communication kernels from launching, even without dependencies. In our experiment, ZeRO-Infinity [57] on  $8\times 1$ -GPU machines under a 100Gbps network only achieves 15% Model FLOPS Utilization (MFU) when training a 175B model.

**SmartSwitch-Enhanced MSDP.** Prior works [66], [67] adopt a SmartSwitch [68], [69] in MRDP to optimize AllReduce collectives by aggregating gradients in the SmartSwitch, thus reducing required network traffic and communication time, as shown in Figure 1(b). However, introducing SmartSwitch further to MSDP can barely reduce the required communication time, as both AllGather (AG) and ReduceScatter (RS) collectives used by MSDP can not be ideally optimized. For AG, SmartSwitch can only reduce the worker sending traffic, while the receiving traffic remains the same and thus bound the overall communication time. Similarly, SmartSwitch can only reduce the worker receiving traffic in RS, while the sending traffic remains the same and thus bound the overall communication time. Existing collective libraries like NCCL cannot efficiently overlap AG and RS due to network bandwidth contention on NIC, thus developers can not run AG and RS concurrently to combine the benefits of SmartSwitch-enabled AG and RS to fully exploit duplex network bandwidth, thus reducing communication time.

**Model-in-Server DP (MiSDP).** Prior works [33], [34], [39] adopt parameter servers (PSs) to minimize network traffic. Each worker only needs to push its gradients to PS and pull parameters from PS, while MiSDP needs PS to aggregate gradients to update parameters. As such, MiSDP reduces one-third of the collective traffic compared to MSDP. However, MiSDP on the one hand requires a large number of additional CPU machines (PSs) to provide adequate CPU compute power and memory bandwidth for heavy gradient aggregation and

<sup>2</sup>Though DeepSeek and Llama use 3D parallelism, including DP, TP, and PP, MSDP is important, especially in the pertaining phase that takes 80%~95% of the total training steps [8], [65]. For example, Llama 3 applies 128-degree MSDP, while employing 8-degree TP and 16-degree PP [64].

<sup>1</sup>In this paper, we refer to a worker as a GPU paired with a NIC.



(a) ZeRO-Infinity: Full aggregation so suffering from interference between collectives and computation on GPU. (b) SwitchML: Partial disaggregation due to only offloading in-network aggregation logic to SmartSwitch. (c) DisDP: Fully disaggregates compute, network (i.e., collectives), and storage (i.e., optimizer) such that GPUs only focus on the computing part to maximize GPU utilization.

Fig. 1. Comparison of different DP architectures. Both ZeRO-Infinity and SwitchML aggregate compute, network, and storage on worker CPU/GPU, leading to severe interference, while DisDP fully disaggregates compute, network, and storage for high GPU utilization.

TABLE I  
COMPARISON BETWEEN DP ARCHITECTURES. DISDP DISAGGREGATES COMPUTE, NETWORK AND STORAGE, SO GPUS ACHIEVE MAXIMUM UTILIZATION.

| System                                   | Interference-Free Network | Traffic-Optimal Collective Topology    | Low-Overhead Optimizer Execution                                                             |
|------------------------------------------|---------------------------|----------------------------------------|----------------------------------------------------------------------------------------------|
| <b>Model-Sharded DP (§II-A)</b>          | No<br>(GPU Contention)    | No<br>(Peer-Based AG/RS)               | Yes (But colocated CPU optimizer could interfere with GPU due to intra-node PCIe contention) |
| <b>SmartSwitch-Enhanced MSDP (§II-B)</b> | No<br>(GPU Contention)    | Yes                                    | Yes (But colocated CPU optimizer could interfere with GPU due to intra-node PCIe contention) |
| <b>Model-in-Server DP (§II-C)</b>        | No<br>(GPU Contention)    | Yes<br>(But only for non-colocated PS) | No (Requiring many additional machines as PSs for aggregation/broadcast)                     |
| <b>DisDP (Ours)</b>                      | Yes                       | Yes                                    | Yes (Requiring only 1 additional machine as scalable PS)                                     |

Adam optimizer. In our experiments, 16 GPU workers that run colocated PS processes require at least 13 additional CPU machines as non-colocated PSs to achieve line rate (Figure 5). On the other hand, MiSDP would still suffer from interference between GPU-centric network communication and computation, which leads to low MFU during training.

**Our Design: DisDP.** Inspired by DeepSeek [63], [70] that argues to use a dedicated network processor for collectives that originally run on GPUs, we propose DisDP, a Fully Disaggregated Distributed Data Parallelism that first disaggregates compute, network (i.e., collectives), and storage (i.e., optimizer) for MSDP to maximize the GPU’s utilization in Figure 1(c). The key idea of DisDP is twofold. First, it offloads collectives to SmartNICs to avoid interference between GEMM kernels and collective kernels (network disaggregation). Second, it offloads the optimizer to a scalable PS where a single PS can serve any number of workers (storage disaggregation). As such, GPUs focus on computation for which GPUs are originally designed, and thus maximizing their utilization, as Table I concludes. To do so, DisDP consists of three innovations:

- **SmartNIC-Managed Interference-Free Collectives.** We propose the first SmartNIC-managed collectives that disaggregate GEMM kernels and collective kernels to completely avoid interference between them.
- **SmartSwitch-Assisted Many-to-One Reliable Protocol.** To address the incast problem introduced by serving many workers with a few PSs, DisDP uses a SmartSwitch to aggregate gradients from workers to the PS and broadcast parameters reversely. However, each worker has to maintain a reliable connection with the PS, and the number of

extra reliability-related packets increases with the number of workers. As the number of workers increases, reliability-related packets soon exhaust the PS network IOPS and degrade the overall network performance. To this end, we propose the SmartSwitch-assisted reliable protocol, which leverages SmartSwitch to reduce the required reliability-related packets.

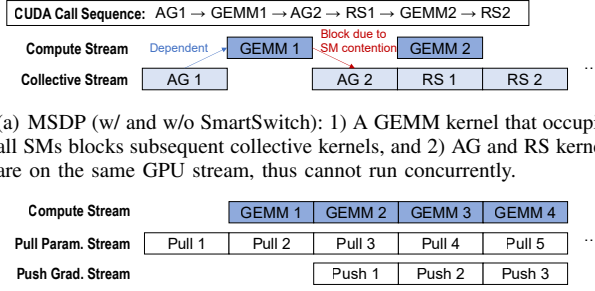
- **Step-Centric Optimizer Pipelining.** Unlike traditional MiSDP that features a large number of PSs, DisDP requires only a single PS to perform optimizer computation for massive workers. The challenge is that traditional layer-centric pipelining would soon exhaust all available CPU threads on the PS, and thus fails to consume line-rate aggregated gradients. Therefore, we propose a step-centric pipelining on the PS to maximize CPU thread usage, so as to consume the aggregated gradients at line rate.

We prototype the SmartNICs with Xilinx U50 FPGA boards. We evaluate DisDP on eight 1×A100 GPU machines and 1 CPU machine under a 100Gbps network. Experiment results show that DisDP with 100Gb/s interference-free in-network collectives achieves 1.17× higher throughput than ZeRO-Offload on the DGX machines (intra-machine GPUs are fully connected with 600GB/s NVLink), while keeping only 60% capex cost of the DGX machines.

## II. BACKGROUND AND MOTIVATION

### A. Issues of Model-Sharded DP (MSDP)

MSDP relies on two optimizations for memory-efficient training: 1) it evenly shards model states and optimizer execution across all workers to aggregate the memory of multiple GPUs, and 2) it offloads the optimizer to CPU memory or



(b) DisDP: 1) SmartNIC-managed collectives do not interfere with GEMM, and 2) contention-free primitives enable concurrent push (for partial gradients) and pull (for on-demand new parameters) operations.

Fig. 2. Computing and collective pattern during the backward stage: MSDP vs. DisDP. A darker color indicates more SMs being used, and white indicates no SMs being used.

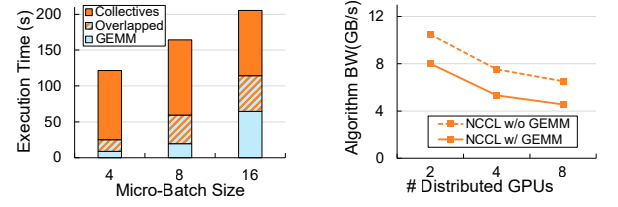
SSDs to further reduce the GPU memory capacity requirement. Compared to MRDP which performs one AllReduce to exchange gradients for each model layer, MSDP requires one RS and two AG collectives for each layer during forward and backward propagation, thus causing heavy collective traffic, e.g., the collectives take  $1.25 \times \sim 4.5 \times$  more time over GPU computation (Figure 3(a)). Current MSDP systems employ collective libraries such as NCCL [71] to execute RS and AG on GPUs. These systems try to pipeline collectives and model computation (i.e., GEMM), assuming that the two processes utilize distinct hardware resources, thereby enabling their overlapping. However, MSDP’s aggregated computation and collectives on GPU cause severe interference, mostly serial execution, thus leading to low GPU utilization, e.g., 15% MFU on eight 1-GPU machines (Figure 16(a)).

Even if we enabled to run GEMM and collective kernels concurrently, it still suffers from severe interference from the following two main issues:

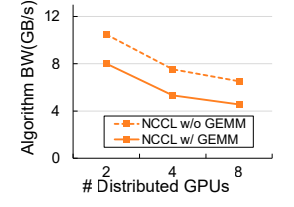
**1, GPU Computing Unit Contention.** GEMM and collective kernels compete for GPU computing units, i.e., streaming multiprocessors (SMs). Collective kernels only require a small portion of SMs (a few percent to 10%) [72], and intuitively, we think it is easy to overlap the GEMM and collective kernels to achieve a high GPU SM utilization. However, due to GPUs’ non-preemptive scheduling policy, a compute-intensive GEMM kernel that occupies all SMs will block subsequent collectives from launching [73], even though a collective kernel has the highest priority. Figure 2(a) shows an example where “GEMM 1” blocks “AG 2” from launching. After “GEMM 1” finishes and “AG 2” starts, “GEMM 2” cannot be launched because it depends on the completion of “AG 2”. Consequently, a collective kernel cannot fully overlap with GEMM kernels.

To illustrate the impact of this interference, we break down the execution time for training OPT-175B on  $8 \times 1$ -GPU machines under a 100Gbps network using ZeRO-Infinity, as shown in Figure 3(a). Up to 65% of the collective execution time cannot overlap with GEMM at the batch size of 16, incurring 41% more execution time than ideal full overlapping.

**2, GPU Memory Bandwidth Contention.** GEMM and

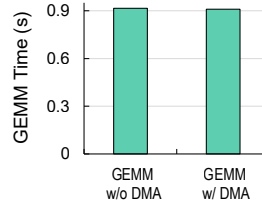


(a) Execution time breakdown in a training iteration.

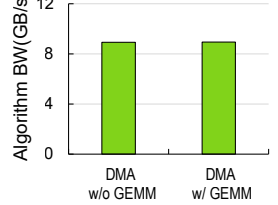


(b) Impact of concurrent GEMM to NCCL AllReduce.

Fig. 3. Issues of MSDP (ZeRO-Infinity).



(a) Impact of concurrent DMA to GEMM execution time.



(b) Impact of concurrent GEMM to GPU DMA bandwidth.

Fig. 4. There is barely interference between GPU GEMM kernel and an external device accessing GPU memory.

collective kernels compete for memory bandwidth and L2 cache, leading to performance degradation of the collective kernels [73], [74]. Typically, we use algorithm bandwidth to characterize the collective performance, which is defined as total data size  $S$  in a collective operation divided by collective execution time  $t$ , i.e., algorithm bandwidth  $BW_{alg} = \frac{S}{t}$ . To show the algorithm bandwidth drop due to contention, we run independent GEMM and NCCL AllReduce kernels on distributed A100 GPUs under 100Gbps network and a typical configuration when training the OPT-175B model, and use CUDA multi-process service (MPS) to statically isolate SMs to avoid the influence of competing SMs.<sup>3</sup> Figure 3(b) shows that the GEMM kernel causes a 30% algorithm bandwidth drop to the concurrent NCCL kernel.

**Finding.** Although the concurrently running GEMM and collectives kernels incur interference regarding SMs and GPU memory subsystem, we observe that there is barely interference when the GEMM kernel is running and the external device is reading/writing GPU memory through the PCIe. To illustrate this, we 1) measure the GEMM execution time with and without a PCIe device (e.g., an FPGA-based NIC) performing DMA requests to read/write GPU memory concurrently; and 2) measure the read/write speed of PCIe devices accessing GPU memory through DMA with/without concurrently running GEMM kernels. Figure 4 indicates there is barely interference between a GEMM kernel and a DMA task of an external PCIe device. Such a finding motivates us to offload the entire collective stack on SmartNIC (i.e., network and compute disaggregation) to avoid interference.

<sup>3</sup>Even though MPS is able to partition SMs, we can not adopt it to eliminate the contention for GPU computing units in actual training because MPS only allow partitioning SMs statically, and thus is incompatible with important features like dynamic parallelism and CUDA graph [75] that LLM training frameworks heavily rely on.

### B. Issues of SmartSwitch-Enhanced MSDP

To relieve the issue of MSDP, a possible way is to implement in-network computing primitives in SmartSwitch [68], [69], [76]–[80] to reduce the collective traffic via in-switch aggregation and broadcasting. However, current SmartSwitch can only provide `AllReduce` collective needed by MRDP, while MSDP relies on AG and RS. Even if we enabled SmartSwitch to run AG and RS, we still identify that the naïve solution has the following two issues.

**1, Unchanged Collective Time for Separate AG and RS.** Considering  $N$  workers performing an AG on a total data size  $S$ . Without a SmartSwitch, each worker sends its  $\frac{S}{N}$  data to every other worker and receives different  $\frac{S}{N}$  data from every other worker [31]. As such, each worker has to send (or receive)  $\frac{(N-1)S}{N}$  network traffic.

If we enabled SmartSwitch to support AG and RS, for AG, SmartSwitch can help each worker to broadcast its  $\frac{S}{N}$  to every other worker, thus each worker only has to send  $\frac{S}{N}$  network traffic, while the receiving traffic remains at  $\frac{(N-1)S}{N}$ . Similarly, for RS, SmartSwitch allows each worker to receive  $\frac{S}{N}$  traffic, while the sending traffic remains at  $\frac{(N-1)S}{N}$ . Therefore, a SmartSwitch can not reduce the network communication time for separate AG and RS because the communication time is decided by the maximum of the sending traffic and the receiving traffic.

**2, Compute-Collective Interference.** If we enabled SmartSwitch to support AG and RS, they still rely on GPU-centric NCCL for data chunk management, as the existing SHARP-based in-switch aggregation stack [81] does. Consequently, they still suffer from computation-collective interference that is identified in Subsection II-A.

### C. Issues of Model-in-Server DP (MiSDP)

To relieve the issue of MSDP, another possible way is to adopt the model-in-server DP (MiSDP) architecture [33], [34], [39], [43] that offloads the model states onto PSs. Each worker performs 1) a `push` operation to send its gradients to PSs, which can aggregate gradients from workers and update the corresponding optimizer states, and 2) two `pull` operations (one in forward stage and one in backward stage) to receive the latest parameters from PSs. Compared to two AG and one RS of MSDP for a layer ( $\frac{3(N-1)S}{N}$  in each direction), MiSDP ( $2S$  receive and  $S$  send) reduces the network traffic by at most one-third. However, existing MiSDP systems push the entire model gradients to PS after GPU finishes computation and then pulls the entire model parameters, so that each GPU has to accommodate the full model parameters. Therefore, they fail to train 100B-scale model that cannot fit in GPU memory. Even if MiSDP were enabled to train large models, we identify that MiSDP solutions still suffer from two issues.

**1, Computation-Collective Interference.** To enable large-model training, MiSDP systems need to use GPU-centric network stacks such as IBGDA (InfiniBand with GPUDirect Async) to enable GPU to directly interact with NICs for higher network bandwidth. Therefore, they would still suffer from

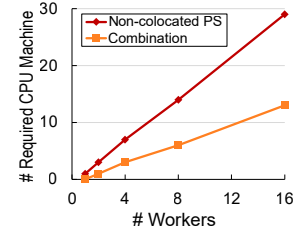


Fig. 5. Overhead of MiSDP: Many extra CPU machines.

interference between GPU computation kernels and collective kernels, as demonstrated in Subsection II-A.

**2, Escalating Hardware Requirement to Aggregate Partial Gradients.** MiSDP requires PS to provide adequate CPU compute power, memory bandwidth, and network bandwidth to perform optimizer, gradient aggregation, and parameter broadcasts. Therefore, a growing number of workers would require many additional CPU machines as PSs to consume their 100Gbps-per-worker partial gradients and produce 100Gbps-per-worker parameters, which is not always acceptable due to their monetary and space costs. To illustrate the machine requirement, we simulate the minimal machine requirement to saturate 100Gbps with different numbers of 1-GPU worker machines, where each GPU machine and each CPU machine has 1×100Gbps NIC. Each server runs a multi-threaded SIMD-optimized loop-unrolled Adam optimizer proposed by ZeRO-Offload [56]. We simulate on both non-colocated PS (PS processes only run on additional CPU machines) and a combination of colocated PS (PS processes run on GPU machines’ spare CPUs) and non-colocated PS. Figure 5 shows that a 16-worker cluster requires 29 additional CPU machines in non-colocated PS configuration and 13 machines in a combined colocated and non-colocated PS configuration to achieve line-rate throughput.

**Finding.** We observe from Figure 5 that the scalability inefficiency of PS comes from more pressure on the memory subsystem of the server due to consuming more partial gradients from an increasing number of workers. This motivates us to offload gradient aggregation and parameter broadcasts to SmartSwitch, which produces reliable aggregated gradients to enable only one CPU machine (storage disaggregation) to accommodate 100B Adam states.<sup>4</sup> Meanwhile, a SmartNIC would disaggregate the network and compute to eliminate the interference issue. This motivates us to use SmartNIC-SmartSwitch co-optimization to fully disaggregate compute, network, and storage, so as to benefit from reduced network traffic of MiSDP while addressing scalability and interference issues, thus achieving high MFU on scalable FSDP training.

## III. DESIGN AND IMPLEMENTATION OF DISDP

### A. DisDP Overview

Inspired by the DeepSeek [63], [70], [82] that argues to use a dedicated network processor for collectives, we propose

<sup>4</sup>ATP [66] only offloads in-network aggregation primitive to SmartSwitch to 1) provide an `AllReduce` primitive, to aggregate gradients to support small model training, rather than LLM; and 2) updates its optimizer states on GPU workers. Thus, ATP suffers from interference from its partial disaggregation.

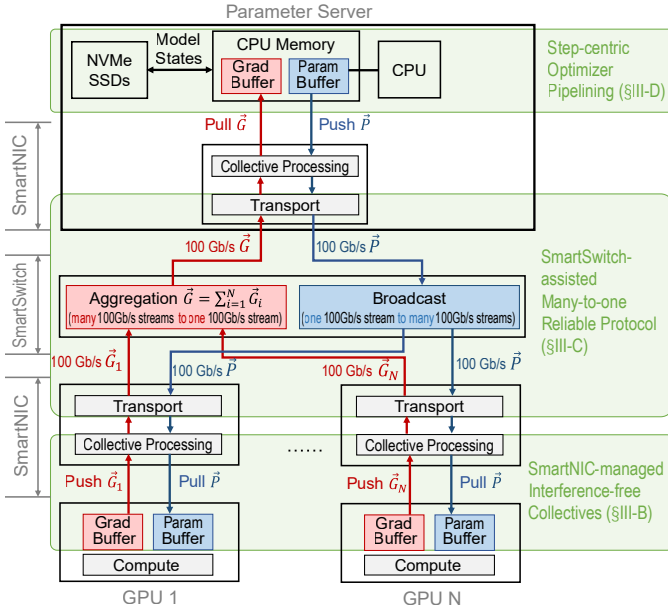


Fig. 6. System overview of DisDP. DisDP offloads collectives and the optimizer to dedicated hardware, such that GPUs only focus on the computing part to maximize their utilization.

DisDP that first fully disaggregates compute, network (i.e., collectives), and storage (i.e., optimizer) to maximize the GPU’s utilization in Figure 1(c). The key idea of DisDP is twofold. First, it offloads collectives to SmartNICs and SmartSwitch to avoid interference between GEMM kernels and collective kernels (network disaggregation). Second, it offloads the optimizer to a scalable PS that can serve any number of workers (storage disaggregation). DisDP leverages SmartSwitch to 1) aggregate line-rate partial gradients from GPU workers to the PS that maintains 100B optimizer states, and 2) broadcast the on-demand parameters reversely. As such, GPUs focus on computation for which GPUs are originally designed, and thus maximize their utilization. However, building DisDP is not trivial, mainly due to three main challenges.

**C1: Offloading collectives to SoC-based SmartNICs cannot saturate network line rate.** A possible solution to avoid SM contention is to offload collectives to SoC-based SmartNICs like BlueField [83]. However, an SoC-based SmartNIC fails to process line-rate packets due to its internal switch link and Arm memory bandwidth bottlenecks.

**C2: Existing reliable protocols would soon exhaust the asymmetrical PS network IOPS.** In DisDP, the single PS has to maintain reliable connections with dozens of workers. Although the gradients/parameter packets can be aggregated/broadcast by the SmartSwitch, other packets to maintain reliable connections would exhaust the limited network IOPS of the PS, resulting in network throughput degradation.

**C3: The optimizer computing would soon exhaust the PS computing power.** Optimizer on the PS needs to perform Adam on the line-rate aggregated gradients and serve line-rate parameters to GPU workers. DisDP exploits SmartSwitch to reduce the number of parameter servers, which results in computing power contention on the PS. Traditional layer-centric

TABLE II  
CORE SOFTWARE APIS OF DISDP.

| API                                  | Description                                                     |
|--------------------------------------|-----------------------------------------------------------------|
| handle_t push(void* buf, size_t len) | Issue a contention-free push request to the SmartNIC.           |
| handle_t pull(void* buf, size_t len) | Issue a contention-free pull request to the SmartNIC.           |
| void wait(handle_t request)          | Block CPU/GPU execution until completion of a specific request. |

pipelining of the optimizer would suffer from the limited CPU threads and lead to degraded serving performance.

**Overall Architecture.** DisDP consists of a series of hardware-software co-designs, as shown in Figure 6. The hardware part mainly consists of three components: 1) a per-GPU FPGA-based SmartNIC that provides an interference-free collective library for worker GPUs; 2) a SmartSwitch that aggregates gradients, broadcasts parameters, and maintains many-to-one reliable connections; 3) a single non-located PS that performs Adam on line-rate aggregated gradients and serves line-rate parameters.

### B. SmartNIC-Managed Interference-Free Collectives

To address **C1**, we propose SmartNIC-managed collectives that perform the entire collectives on FPGA-based SmartNICs, while GPUs only need to perform the model’s forward and backward computation. Each SmartNIC is paired with a GPU, as shown in Figure 6.

**Software API.** DisDP provides a two-sided software API for both workers and the PS, so as to enable the PS to prefetch parameters, thus reducing the latency of workers pulling parameters from the PS. Table II lists the core APIs. During initialization, workers and the PS’s CPU invoke the `register_buf` function to enable direct data movement between the application buffer and SmartNICs. At runtime, DisDP provides *contention-free* push and pull to workers and the PS to enable concurrent push and pull calls to fully exploit the network bandwidth, as shown in Figure 2(b). Each worker invokes a push request to the SmartNIC to send partial gradients of a whole model layer from GPU memory, meanwhile, the PS invokes a pull request to the SmartNIC to receive gradients aggregated by SmartSwitch to CPU memory. Similarly, the PS issues a push request to send parameters of a whole layer from CPU memory, and each worker issues a pull request to receive parameters broadcast by SmartSwitch to GPU memory. Both push and pull return a handle, so workers and the PS can wait until the completion of the request via the wait function with the handle.

*1) Naïve Solution: SoC-Based SmartNIC-Managed Collective Library:* To avoid interference, a straightforward solution is to offload collectives to SoC-based SmartNICs [84]–[87] (e.g., Nvidia BlueField-3 [83], [88]), which allows C/C++ software-programming. However, existing off-the-shelf SmartNICs suffer from two severe issues, which are also reported by prior work [89].

**Issue 1: Off-path SmartNIC’s Internal Switch Link Contention.** Off-path NICs dominate the data center NIC market due to their ease of integration and full operating system

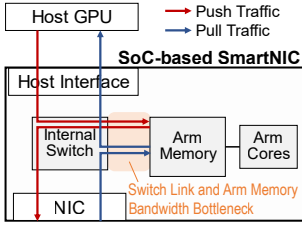


Fig. 7. Push/pull traffic inside a worker-side SoC-based SmartNIC.

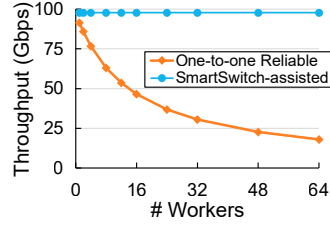
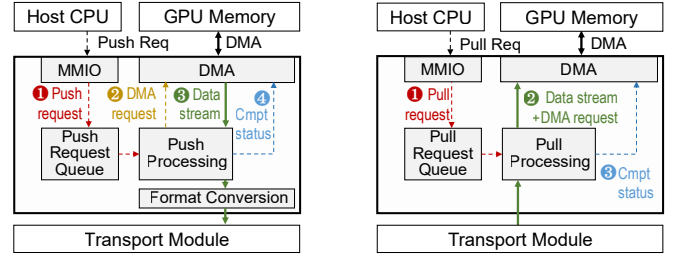


Fig. 8. Throughput under different reliable protocols.

support. In an off-path SmartNIC, the host interface, NIC, and Arm are connected by an internal switch, as Figure 7 shows. The Arm core processes traffic between host and network in a lookaside manner. In the case of model training, both push and pull traffic first goes from the internal switch to Arm for processing, then from Arm back to the internal switch. However, this would result in severe Arm-Switch PCIe link contention when serving bi-directional network traffic at line-rate. Arm-switch bandwidth is usually only slightly higher than network bandwidth, where push and pull throughput would nearly be halved due to the contention. For example, BlueField-2’s per-direction network bandwidth is 200Gbps, concurrent push/pull require 400Gbps per-direction Arm-Switch bandwidth to fully saturate the network bandwidth, while the actual per-direction Arm-Switch is only 250Gbps.

**Issue 2: Arm Memory Bandwidth Bottleneck.** Even if there were no switch link contention introduced by off-path architecture, push/pull performance would still be constrained by the SoC-based SmartNIC’s Arm memory. As traffic is staged to Arm memory rather than being served by cache due to the large working set size (which matches DisDP’s case), push traffic first goes from host interface to Arm memory, then from Arm memory to network, and pull traffic first goes from network to Arm memory, then from Arm memory to host interface, which incurs  $2\times$  memory access for per-direction traffic, as Figure 7 shows. However, an SoC-based SmartNIC lacks adequate memory bandwidth for this access pattern. For example, an off-the-shelf BlueField-2 SmartNIC requires 800Gbps memory bandwidth to serve 200Gbps packets, while it only provides 204.8Gbps theoretical memory bandwidth [90]. As a result, we only achieve 20% network link utilization when evaluating push/pull in a real BlueField-2 SmartNIC. This memory bandwidth constraint still holds for newer-generation BlueField-3, which requires 1600Gbps memory bandwidth to achieve 400Gbps line-rate throughput, while it only provides 716.8Gbps theoretical memory bandwidth [91]. Due to the constraints of power, chip scaling, and form factor, this issue is unlikely to be resolved by near-future SoC-based SmartNICs [92], [93].

2) *Our Solution: FPGA-Based SmartNIC-Managed Collective Library:* To address the memory bandwidth issue, we propose the *FPGA-based SmartNIC-managed collective library* that performs the collectives from GPU/CPU on FPGA-based SmartNICs [94]–[110]. The key insight is that an FPGA-based SmartNIC adopts an on-path architecture that processes packets in hardware pipelines rather than a lookaside SoC,



(a) Pushing gradients

(b) Pulling parameters

Fig. 9. Collective processing module of a worker.

which eliminates the internal switch bandwidth contention. Meanwhile, an FPGA-based SmartNIC allows explicitly storing packets between pipeline stages in on-chip SRAM instead of forcing packets to off-chip DRAM, which addresses the memory bandwidth constraints. Therefore, they can serve line-rate packets. To this end, we design a *collective processing module* on each FPGA-based SmartNIC to process the collectives. In the following, we describe the detailed procedure for handling push/pull calls.

**Handling Contention-Free Push and Pull.** Figure 2(b) illustrates the execution pattern of DisDP. Like GPU streams, DisDP’s SmartNIC provides two independent SmartNIC streams for pull and push calls, respectively, and each stream has its own hardware modules to avoid interference between push and pull requests.

To achieve this, DisDP provides separate push/pull processing units for push and pull primitives. Figure 9 shows a simplified flow of a worker’s collective processing module handling requests, where push and pull are handled by different processing units to execute concurrently. When the host CPU calls push or pull, the software library allocates a request handle and then passes the request with the handle pointer to the SmartNIC via MMIO. Then, the request is enqueued in a push/pull request queue, such that a request called first finishes first. Next, the push/pull processing unit accepts the request from the queue and issues a DMA request to access GPU memory. The push/pull processing unit processes data from/to GPU memory in a pipelined manner to enable concurrent DMA and network data transport. Upon completion of the DMA procedure, the push/pull processing unit writes the execution status to the request handle to finalize the host wait function.

**Enabling Direct Access to GPU Memory.** We follow the existing works [111], [112] to enable SmartNIC to directly access the GPU memory via GPU virtual addresses.

**Format Conversion Unit.** Large-scale model training typically produces gradients in fp16 or bf16 formats [113]. However, current SmartSwitches do not support floating-point arithmetic. To this end, we follow the format conversion strategy of SwitchML [114] and integrate the strategy into the format conversion unit that is hardcoded in SmartNICs.

### C. SmartSwitch-Assisted Many-to-One Reliable Protocol

To address C2, we propose a SmartSwitch-assisted reliable protocol that uses SmartSwitch to aggregate ACKs from

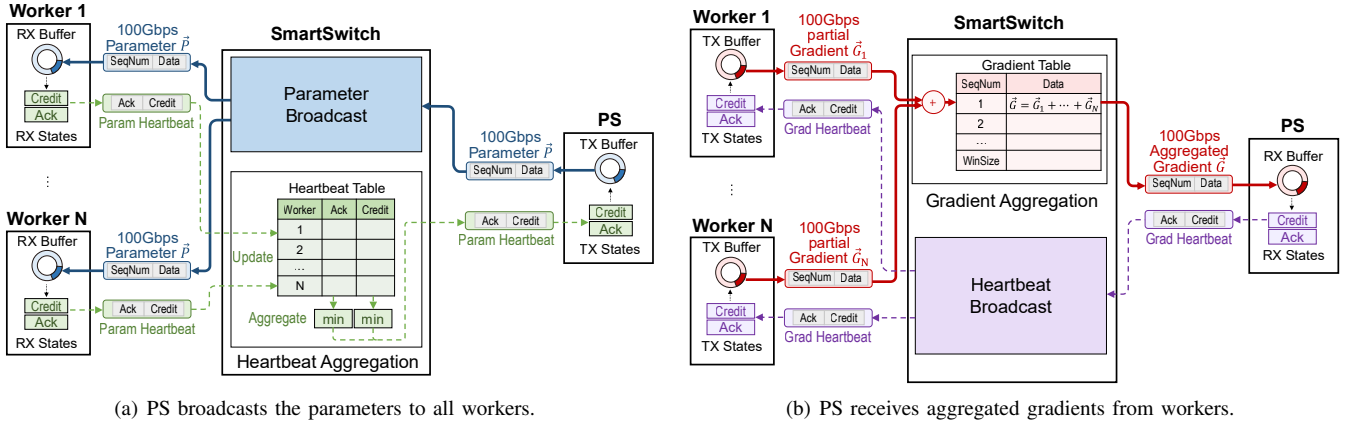


Fig. 10. SmartSwitch-assisted many-to-one reliable protocol that reduces the acknowledgement traffic between the SmartSwitch and the PS.

workers to the parameter server and broadcast ACKs reversely. A reliable protocol requires 1) flow control that ensures the sender sends data at the same rate as the receiver receives data to prevent a fast sender from overwhelming the receiver, and 2) reliability assurance that the primitives can detect and recover packet loss or data error.

1) *Naïve Solutions: Establishing One-to-One Reliable Connections:* A straightforward solution is to establish a one-to-one reliable connection between the SmartSwitch and each worker/PS. However, this solution requires the switch to act as an endpoint executing heavy reliable protocol, which is impractical due to insufficient hardware resources. Specifically, a Tofino SmartSwitch processes a packet in a pipeline with  $\leq 20$  hardware stages [68], whereas a reliable RDMA or TCP packet requires more than 50 hardware stages to process.

Another solution is to establish a one-to-one reliable connection between each worker and the PS, such that the SmartSwitch only forwards the acknowledgment packets without any processing. However, this method has two issues: First, a PS that executes reliable protocols with many (e.g., 64) workers simultaneously incurs high implementation complexity to SmartNICs; Second, existing TCP and RDMA adopt per-packet acknowledgment, so the PS must receive many acknowledgment packets from workers after sending one data packet. Consequently, massive acknowledgment packets occupy non-negligible network bandwidth, lowering the throughput. To demonstrate this, we simulate the maximum achievable data throughput with different numbers of workers under 100Gbps network. The orange line in Figure 8 shows that the throughput drops to 30Gbps with 32 workers, and only 18Gbps with 64 workers.

2) *Our Solution: SmartSwitch-Assisted Many-to-One Reliable Protocol:* To this end, we propose a *SmartSwitch-assisted many-to-one reliable protocol* that 1) uses periodical heartbeat packets to reduce the acknowledgment traffic between workers and the PS, and 2) uses SmartSwitch to aggregate many heartbeat packets from workers to one heartbeat packet to the PS, and broadcast one heartbeat packet from PS to many heartbeat packets to workers. Thus, the PS only sends and receives one acknowledgment packet for each heartbeat cycle, regardless of the number of workers to maintain good scalability. The

blue line in Figure 8 shows that our SmartSwitch-assisted protocol achieves a higher data throughput than one-to-one reliable connections, and the throughput does not drop as the number of workers scales.

**Packet Transport Procedure.** Figure 10 illustrates the packet transport procedure of DisDP. The SmartSwitch follows SwitchML [114] to broadcast parameters and aggregate gradients. Additionally, each worker and the PS periodically send heartbeat packets containing two connection states: *Ack* that indicates the next expected sequence number, and *Credit* field that indicates the maximum sequence number the heartbeat sender can accept. The SmartSwitch broadcasts the gradient heartbeat from the PS to all workers. Meanwhile, it maintains a heartbeat table to record the latest parameter heartbeat from each worker, and periodically performs minimum aggregation to *Ack* and *Credit* in the table and sends a heartbeat packet containing the aggregated states to the PS.

For flow control, a worker/PS ceases sending packets once it sends a packet with a sequence number equal to its TX *Credit* value, and resumes only after TX *Credit* value is updated by the heartbeat packets; for reliability assurance, if there are unacknowledged packets and a sender TX state's *Ack* has not increased for the Retransmission Time Out (RTO) threshold, the sender resends packets starting with the sequence number *Ack*. The SmartSwitch simply forwards any resent packet to the receiver. We follow industry practices [?] to set the RTO threshold to  $\sim 250$  us.

#### D. Step-Centric Optimizer Pipelining

Thanks to the SmartSwitch-assisted many-to-one reliable protocol that provides reliable aggregated gradients, our PS needs to perform Adam optimizer on the line-rate aggregated gradients and provide on-demand parameters to GPUs. However, it is not trivial to achieve. The main challenge is that the size of model states is way larger than the host memory size, so it is generally assumed that a 100B model needs many CPU servers to implement the corresponding CPU optimizer [43]. To this end, to address **C3**, we propose the *step-centric pipelining technology* to 1) allocate PS's CPU threads by optimizer steps, instead of by layers, and 2) deeply pipeline SSD accesses, CPU Adam, and collectives to enable an out-

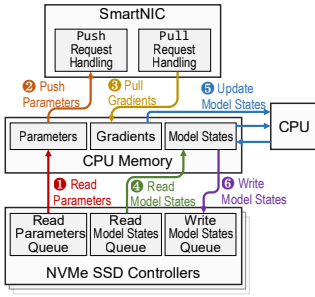


Fig. 11. Data path of the in-network optimizer during the backward stage.

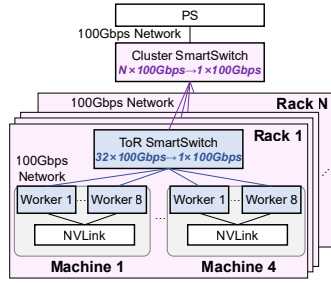


Fig. 12. Multi-rack topology of DisDP.

TABLE III

RESOURCE REQUIREMENTS FOR ADAM OPTIMIZER.

| Network Thpt | Stage | Compute FLOPS | Mem Bandwidth (Read+Write Total) | SSD Bandwidth (Per Direction) |
|--------------|-------|---------------|----------------------------------|-------------------------------|
| 100Gbps      | Fwd   | 0             | 23.3 GB/s                        | 11.6 GB/s                     |
|              | Bwd   | 99.0 GFLOPS   | 349 GB/s                         | 81.4 GB/s                     |
| 200Gbps      | Fwd   | 0             | 46.6 GB/s                        | 23.3 GB/s                     |
|              | Bwd   | 198 GFLOPS    | 698 GB/s                         | 163 GB/s                      |

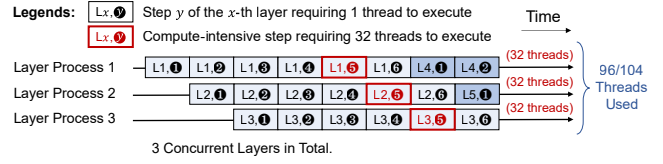
of-core optimizer to consume 100Gbps aggregated gradients. Figure 11 illustrates a simplified data path of our out-of-core optimizer. In the following, we present the resource requirement of a training iteration that consists of two stages: forward and backward, followed by the detailed design of *step-centric pipelining technology*.

**Forward Stage.** During this stage, the optimizer needs to provide on-demand parameters to GPUs. It undergoes two steps: ① Reading 2-byte parameters from the SSDs to the CPU memory, and ② pushing 2-byte parameters from CPU memory to SmartNIC. Therefore, providing 100Gbps (11.6 GB/s) on-demand parameters to GPUs requires 23.3 GB/s memory bandwidth and 11.6 GB/s SSD bandwidth in Table III.<sup>5</sup>

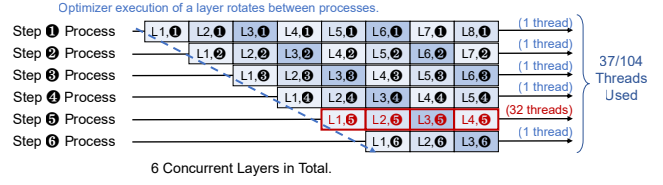
**Backward Stage.** During this stage, the optimizer provides on-demand parameters to GPUs and performs the Adam operation on the aggregated gradients. It undergoes the two steps (① and ②) and four additional steps: ③ pulling 2-byte gradients from the SmartNIC to the CPU memory, ④ reading 12-byte model states from SSDs to the CPU memory, ⑤ performing the **compute-intensive** CPU Adam, where the CPU reads 2-byte gradients and 12-byte model states from the memory, performs 17 floating-point operations to update model states [56], and writes 12-byte updated model states and 2-byte parameter copy to the memory, and ⑥ writing 12-byte updated model states and 2-byte parameter copy back to SSDs. This stage requires 99 GFLOPS computation, 349 GB/s memory bandwidth, and 81.4 GB/s SSD bandwidth.

**Naïve Solution: Layer-Centric Pipelining.** A straightforward method to pipeline the optimizer steps is *layer-centric pipelining*, which assigns a fixed number of CPU threads for each model layer to sequentially execute the optimizer steps. This method naturally pipelines the steps by concurrently executing the optimizer across multiple layers. However, this approach incurs an issue: Different steps in the optimizer require varying numbers of CPU threads to saturate. In par-

<sup>5</sup>Providing 200Gbps on-demand parameters doubles the memory and SSD bandwidths.



(a) Layer-centric pipelining: Limited parallelism causes pipeline bubbles.



(b) Our step-centric pipelining: Fully pipelines the optimizer steps.

Fig. 13. Pipelining of collectives, SSD IO, and CPU Adam during the backward stage.

TABLE IV

RESOURCES PROVIDED BY DIFFERENT CPUS.

| CPU           | Compute FLOPS | Mem Bandwidth | PCIe Bandwidth |
|---------------|---------------|---------------|----------------|
| 5320 (Gen 4)  | 1.83 TFLOPS   | 375 GB/s      | 246 GB/s       |
| 6730P (Gen 5) | 2.56 TFLOPS   | 819 GB/s      | 678 GB/s       |

ticular, CPU Adam (Step ⑤) requires 32 threads on Intel Xeon 5320 CPU to consume aggregated gradients at line rate, while the remaining steps only require 1 CPU thread to saturate. Consequently, layer-centric pipelining requires 32 CPU threads to execute each model layer, thereby limiting the overall achievable parallelism in Figure 13(a).

**Our Solution: Step-Centric Pipelining.** To address the limited parallelism issue, we propose *step-centric pipelining* that allocates CPU threads to each optimizer step. Each step executes one layer at a time and a layer rotates from the first step to the last. This technology allows us to allocate more threads to the compute-intensive Step ⑤ and fewer threads to the remaining steps, as Figure 13(b) shows, so as to enable full pipelining of optimizer steps with a small number of CPU threads, e.g., 37 threads in Figure 13(b).

**100B Model Optimizer in a Scalable PS.** Table IV shows the compute power, memory bandwidth, and PCIe bandwidth of two commodity machines, each with two CPUs: Intel Xeon 5320 (PCIe Gen 4) and 6730P (PCIe Gen 5). We observe that 6730P (or 5320) machine can satisfy more than 200Gbps (or 100Gbps) network, in case the machine provides sufficient SSD bandwidth, e.g., with 12 SSDs. We conclude that a single scalable PS is sufficient to concurrently provide on-demand parameters and perform the Adam operation on the aggregated gradients from any number of workers at line rate.

**Supporting Multi-Rack.** DisDP supports training on multi-rack clusters by hierarchical switches in a topology shown in Figure 12. Each rack employs a ToR SmartSwitch to aggregate gradients from its workers to compute partial aggregated gradients. A cluster SmartSwitch aggregates partial aggregated gradients from the ToR SmartSwitches and forwards the fully aggregated gradients to the PS. The parameters are broadcast in a reverse hierarchical manner. Supporting more workers only needs a deeper hierarchy of SmartSwitches.

TABLE V  
MODELS FOR EVALUATION. CUSTOM MODELS KEEP THE SAME ARCHITECTURE AS OPT MODELS WITH RANDOM PARAMETERS.

| Model       | #Transformer Blocks | #Head | Hidden Dimension |
|-------------|---------------------|-------|------------------|
| OPT-1.3B    | 24                  | 32    | 2048             |
| OPT-2.7B    | 32                  | 32    | 2560             |
| OPT-6.7B    | 32                  | 32    | 4096             |
| OPT-13B     | 40                  | 40    | 5120             |
| OPT-30B     | 48                  | 56    | 7168             |
| OPT-66B     | 64                  | 72    | 9216             |
| Custom-175B | 96                  | 96    | 12288            |
| Custom-276B | 112                 | 112   | 14336            |
| Custom-505B | 124                 | 144   | 18432            |
| Custom-1.0T | 172                 | 172   | 22016            |

#### IV. EVALUATION

##### A. Experimental Setup

**Workloads.** We choose OPT models [115] and our custom models of different sizes listed in Table V. Our results can be generalized to other LLMs such as Llama [8] since their computation/collective patterns are similar.

In each of the evaluation experiments, we select the largest model size that the baselines can train under the corresponding configuration, which is 175B for Subsection IV-C, IV-E, and end-to-end breakdown and MPS comparison in Subsection IV-D, 30B for SHARP comparison in Subsection IV-D, 13B for Subsection IV-G. The sequence length is set to 1024 for all experiments.

**Evaluated Cluster.** We evaluate DisDP and baselines on an 8-worker cluster, each worker with dual Intel Xeon Silver 4214 CPUs, 256 GB DDR4 memory, 1× NVIDIA A100 40GB GPU, and system-specific network and SSDs. All machines are connected by 100Gbps network. We will introduce the network and SSD settings of DisDP and baselines below.

**DisDP’s Configurations.** We evaluate DisDP on the cluster with 8 machines (each with 1 GPU and 1 SmartNIC connected to a SmartSwitch [116]) plus a PS. The PS equips dual Intel Xeon Gold 5320 CPUs, 12 SSDs, and a SmartNIC. We implement SmartNICs on Xilinx Alveo U50 FPGAs. Tables VI and VII show the resource consumption of SmartNICs and the SmartSwitch logic. DisDP enables activation checkpointing [117], [118] and bf16 training [113].

**Baselines.** We use four systems as our baselines.

The first baseline is ZeRO-Infinity [57], an MSDP system that distributes the model states in workers’ SSDs. We evaluate ZeRO-Infinity on the cluster with 8 machines, and 1 Mellanox SN2700 switch [119]. Each machine features 2 SSDs (16 in total<sup>6</sup>), 1 GPU, and 1 CX-5 NIC connected to the switch. We run ZeRO-Infinity on DeepSpeed 0.9.3 [120] and NCCL 2.20.5 with activation checkpointing and bf16 training.

The second baseline is ZeRO-Offload [56], which is used as an alternative to ZeRO-Infinity for experiments on the NVLink machine (Subsection IV-G), because the NVLink machine we rent does not support plugging ad-hoc SSDs and thus

<sup>6</sup>ZeRO-Infinity equips 4 more SSDs than DisDP because our SSDs provide different I/O bandwidth on PCIe Gen 3 worker machine of ZeRO-Infinity and Gen 4 PS machine of DisDP. Our configuration ensures both systems have 26 GB/s aggregated I/O bandwidth per direction with 1:1 mixed read/write.

TABLE VI  
HARDWARE RESOURCE CONSUMPTION OF SMARTNIC.

| LUT             | FF              | BRAM           | URAM           |
|-----------------|-----------------|----------------|----------------|
| 135K<br>(15.5%) | 225K<br>(12.9%) | 354<br>(26.3%) | 128<br>(20.0%) |

TABLE VII  
HARDWARE RESOURCE CONSUMPTION OF SMARTSWITCH.

| Stage         | MAT           | TCAM            | VLIW              | Register      | SRAM              |
|---------------|---------------|-----------------|-------------------|---------------|-------------------|
| 11<br>(91.7%) | 99<br>(51.6%) | 111B<br>(28.9%) | 1.64Kb<br>(12.0%) | 37<br>(77.1%) | 48 MiB<br>(39.5%) |

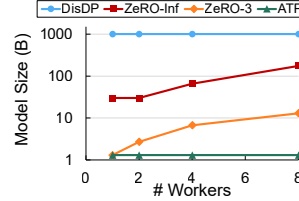


Fig. 14. Maximum trainable model size.

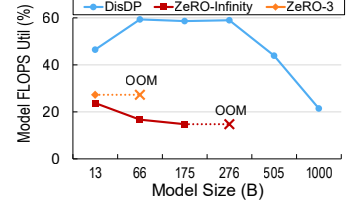


Fig. 15. Maximum MFU on different models.

we cannot achieve high performance from ZeRO-Infinity that relies on more SSDs to efficiently train a large model. We run ZeRO-Offload with the same configuration as ZeRO-Infinity, except that the model states are stored in CPU memory rather than SSDs.

The third baseline is ZeRO-3 [50], which adopts MSDP but keeps the model state shard in GPU memory instead of CPU memory or SSDs. We run ZeRO-3 on the same cluster and with the same configuration as that in ZeRO-Infinity, except that model states are kept in GPU memory.

The fourth baseline is ATP [66], a SmartSwitch-enhanced MRDP system that provides coupled push-pull primitives (same semantics as AllReduce). We run ATP on the cluster with 8 machines, 8 GPUs, 8 CX-5 NICs, and 1 SmartSwitch. Each machine has 1 GPU and 1 NIC connected to the SmartSwitch. We run ATP on PyTorch 1.9.1 [121] with activation checkpointing and bf16 training.

##### B. Maximum Trainable Model Size

We compare the maximum trainable model sizes between DisDP and baselines when varying the number of workers, as shown in Figure 14. We make three observations.

First, DisDP allows the constant, maximum trainable model size 1T that is bounded by the size of a single layer to fit in GPU memory, because DisDP disaggregates storage so that both worker CPU and GPU do not have to accommodate the entire model state shard. In contrast, ZeRO-Infinity with 8 worker machines can train only a 175B model, whose size is smaller than that of DisDP, because the sharded model states require each worker to prepare auxiliary temporary buffers on the CPU memory for GPU-CPU-SSD communications, and thus limits the maximum trainable model size. Second, when employing an increasing number of workers, ZeRO-3 and ZeRO-Infinity increase the trainable model size, because they rely on aggregated GPU, CPU memory, and NVMe storage of worker machines to execute the distributed optimizer. However, its maximum trainable model size cannot exceed that

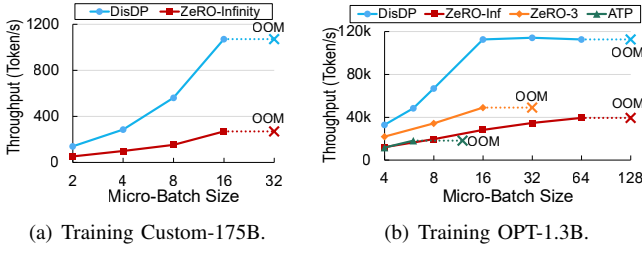


Fig. 16. Throughput comparison when training on 8 distributed GPUs with different micro-batch sizes.



Fig. 17. Execution breakdown comparison.

of DisDP’s to accommodate a single layer on GPU memory. Third, ATP is only able to train a 1.3B model, because it is based on MRDP which replicates the entire model states across all GPUs, bounding its maximum trainable model size.

### C. End-to-End Throughput/MFU Comparison

**Throughput w.r.t. Batch Size.** Figure 16(a) shows the comparison of DisDP and ZeRO-Infinity on the 175B model. DisDP achieves up to 134 token/s,  $3.98\times$  larger than ZeRO-Infinity. The execution breakdown of an iteration in Figure 17 shows the sources of performance gain: 1) DisDP achieves a significantly shorter time for the forward and the backward stages than ZeRO-Infinity because the heavy collectives bottlenecks in the two stages of ZeRO-Infinity. In contrast, DisDP eliminates the interference between compute and collectives by performing collectives on SmartNICs, and uses SmartSwitch-assisted reliable protocol to further reduce the collective traffic, thus achieving high GPU utilization. 2) The PS additionally hides the CPU optimizer behind the GPU backward stage. Notably, DisDP has more benefits as model size increases.

Figure 16(b) shows a comparison of DisDP and the baselines on the OPT-1.3B model. DisDP achieves up to 14.1K tokens/s, which is  $2.90\times$ ,  $2.33\times$ , and  $6.28\times$  larger than ZeRO-Infinity, ZeRO-3, and ATP at their peak throughput respectively. We make two observations.

First, DisDP achieves even higher throughput than ZeRO-3 and ATP, though DisDP stores the model states in slow SSDs in PS, while ZeRO-3 and ATP accommodate the model states in fast GPU memory, because DisDP completely eliminates compute-network interference, while the baselines do not due to partial aggregation of compute, network, and storage.

Second, DisDP’s throughput saturates when the micro-batch size is larger than 16. Figure 18(b) shows the source of the saturation: DisDP’s computation time within an iteration grows linearly as micro-batch size grows, and is roughly the same as collective time at micro-batch size of 16. Since

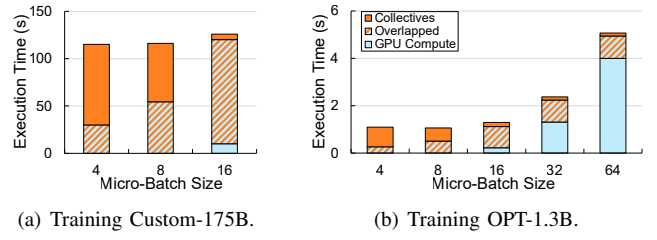


Fig. 18. Breakdown of compute and collectives running DisDP.

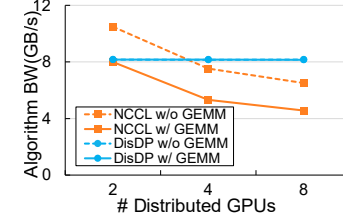


Fig. 19. Algorithm bandwidth of different collective primitives.

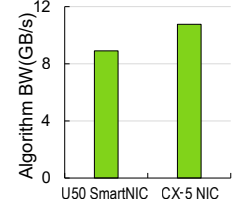


Fig. 20. DMA capability comparison.

collective traffic is decided only by the model, the collective time within an iteration does not change when the micro-batch size changes. Thus, collectives are almost fully hidden behind computation at micro-batch sizes larger than 16 when there is no interference, and the training time is bounded by GPU computation instead of network bandwidth. Therefore, the saturated throughput at micro-batch sizes greater than 16 represents the maximum throughput provided by the GPUs.

**MFU under Different Model Sizes.** Figure 15 shows the end-to-end MFU of DisDP and baselines on different models at their maximum trainable micro-batch sizes. We observe that 1) DisDP achieves at least  $2.04\times$  and  $2.34\times$  MFU over ZeRO-3 and ZeRO-Infinity when model size varies from 13B to 175B, and 2) DisDP still maintains high MFU (59%) when training on a 276B model. Even though the MFU significantly drops on 505B and 1T models due to the small trainable micro-batch size (16 and 8, respectively) per GPU, DisDP still achieves comparable MFU on a 1000B model to ZeRO-Infinity on a 13B model.

### D. Effect of SmartNIC-Managed Collectives

To validate the effect of the SmartNIC-managed interference-free collectives, we run DisDP push and pull and NCCL AllReduce to aggregate data with and without concurrent GPU GEMM kernels (experimental setup in Subsection II-A). Figure 19 shows the algorithm bandwidth of DisDP and NCCL. We make three observations.

First, the concurrent GEMM does not incur overhead to DisDP because DisDP offloads the collectives from GPU to SmartNIC, thus minimizing the interference between collectives and GEMM. Therefore, DisDP with concurrent GEMM achieves 2%, 35%, and 44% higher algorithm bandwidth than that of NCCL on 2, 4, and 8 distributed GPUs, respectively. Second, DisDP increases the algorithm bandwidth by 8% and 20% on 4 and 8 GPUs without concurrent GEMM compared to NCCL because the push-pull primitives reduce the collective traffic by at most half compared to NCCL AllReduce. Third,

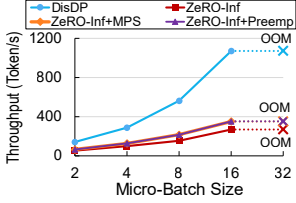


Fig. 21. DisDP vs. simulated ZeRO-Infinity w/o computing unit contention.

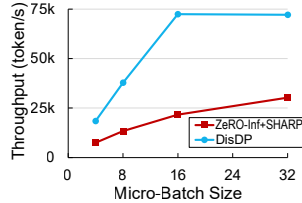


Fig. 22. DisDP vs. ZeRO-Infinity w/ SHARP.

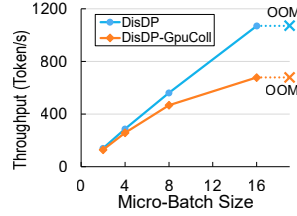


Fig. 23. Effect of interference-free collectives.

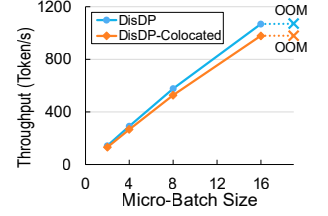


Fig. 24. Effect of many-to-one reliable protocol.

NCCL achieves 28% higher algorithm bandwidth than DisDP on 2 GPUs without concurrent GEMM. This is because the U50 SmartNIC we use has weaker DMA capability (i.e., supporting fewer outstanding PCIe transactions) than a CX-5 NIC. To illustrate this, Figure 20 shows the bandwidth of U50 SmartNIC and CX-5 NIC performing bidirectional DMA to GPU memory. We observe that a CX-5 NIC achieves 24% higher bandwidth than the U50 SmartNIC. Therefore, we expect DisDP to have a similar algorithm bandwidth to NCCL on SmartNICs with better DMA capability.

**Effect on End-to-End Training.** We break down an iteration of DisDP for training Custom-175B, whose experimental setup is the same as Subsection II-A. The result is in Figure 18(a) and shows that 1) DisDP almost fully overlaps computation and collectives, while ZeRO-Infinity (Figure 3(a)) does not; and 2) DisDP reduces the collective time by 20%.

**Comparison to ZeRO-Infinity with Improved GPU Scheduling.** To further break down the effect of GPU computing unit contention in compute-collective interference, we conduct a simulation comparison between DisDP and ZeRO-Infinity with two hypothetical GPU scheduling optimizations: ZeRO-Infinity with MPS (ZeRO-Inf+MPS) that isolates GPU SMs, and ZeRO-Infinity with GPU SM preemption (ZeRO-Inf+Preemp). We simulate the two baselines' collectives by inserting virtual collectives with predefined bandwidth (Figure 19) to ZeRO-Infinity's computation traces, and simulate ZeRO-Inf+MPS's computation kernel by adding MPS overhead coefficient to computation traces.

Figure 21 shows the systems' throughput for training Custom-175B. We make two observations. First, both ZeRO-Inf+MPS and ZeRO-Inf+Preemp increase ZeRO-Infinity's throughput by 30%, and their throughput are almost overlapped (only 0.1%~0.4% throughput differences), because both optimizations remove GPU computing unit contention, thus achieving similar effects. Second, ZeRO-Inf+MPS and ZeRO-Inf+Preemp only achieve 32.7% and 32.8% of DisDP's throughput, because these optimizations do not address GPU memory bandwidth contention between GEMM and collective kernels demonstrated in Subsection II-A, so there is still interference between computation and collectives. In contrast, DisDP's disaggregated design fully overlaps computation and collectives. In conclusion, we need full disaggregation, rather than a better GPU computing unit scheduling.

**Comparison to ZeRO-Infinity with SHARP.** To eliminate the impact of in-network aggregation, we compare DisDP and ZeRO-Infinity with NVIDIA SHARP [122], [123], so that the main difference between the two systems is GPU/SmartNIC-

managed collectives. Because we only have  $4 \times 100\text{Gbps}$  CX-6 NICs that SHARP requires, we run ZeRO-Infinity on 4 workers, each has  $1 \times \text{GPU}$ ,  $4 \times \text{SSDs}$  (to ensure SSD bandwidth is the same as DisDP),  $1 \times \text{CX-6 NICs}$  connected to an Infiniband switch [124]. Figure 22 shows their throughput when training on the OPT-30 B model.

We observe that DisDP achieves  $2.38 \sim 3.35 \times$  throughput over ZeRO-Infinity+SHARP, because ZeRO-Infinity+SHARP still suffers from GEMM-collective interference due to partial disaggregation of compute and network. In conclusion, we need full disaggregation from DisDP, rather than only using in-switch aggregation.

### E. Ablation Study

1) *Interference-Free Collectives Ablation:* To assess the contribution of the interference-free collectives, we compare DisDP with DisDP-GpuColl that uses a GPU kernel to issue push/pull requests to SmartNIC and poll completion, so that GEMM and push/pull kernels compete for GPU computing units. Since the collectives are coupled with many-to-one protocol, we cannot actually run DisDP-GpuColl, so we simulate the throughput of DisDP-GpuColl training Custom-175B on our 8-worker cluster, as Figure 23 shows.

We make two observations. First, DisDP achieves  $1.58 \times$  maximum throughput over DisDP-GpuColl, because DisDP-GpuColl fails to fully overlap computation and network communication due to computing unit contention. Second, when a small micro-batch size is (2 and 4) leads to a trivial throughput gain because a small micro-batch size takes much shorter computation time compared to collectives (shown in Figure 18(a)), so the benefit of compute-network overlapping is not significant. However, the performance gain at large micro-batch size (16) is significant, because compute and network then take roughly the same time and are fully overlapped, thus bringing the most significant benefit.

2) *Many-to-One Reliable Protocol Ablation:* To examine the effect of the many-to-one reliable protocol, we compare DisDP with DisDP-Colocated, a baseline that uses conventional colocated PS architecture, rather than DisDP's scalable PS to execute optimizer. Since DisDP's many-to-one protocol is coupled with SmartNIC-centric collectives, we cannot actually run DisDP-Colocated, so we simulate DisDP-Colocated by inserting virtual collectives to DisDP's compute traces.

Figure 24 shows the simulated throughput of two implementations on the Custom-175B model. DisDP achieves  $1.10 \times$  throughput over DisDP-Colocated at different batch sizes, because colocated PS incurs both worker and PS traffic

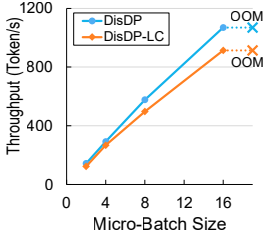


Fig. 25. Effect of step-centric pipelining.

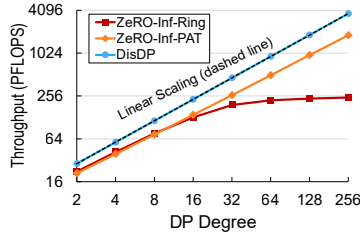


Fig. 26. Scaling of DisDP: ASTRA-sim simulation on 175B model with TP8, PP16, and varying DP degree.

on the same machine, leading to heavier collective traffic during backward propagation. During backward propagation, a SmartNIC has 1 worker push and 1 worker pulls for each layer, incurring  $S$  send receive. Additionally, the SmartNIC has 1 PS push and 1 PS pull, incurring  $\frac{S}{N}$  send receives on each PS. Therefore, DisDP-Colocated has  $(1 + \frac{1}{N})$  send receives in total, which is  $1.13\times$  collective traffic than DisDP on 8 workers (1 push and 1 pulls during backward stage for worker, which is  $S$  send receives. Similarly, PS also has  $S$  send receives) on each NIC, leading to longer collective time. At micro-batch sizes up to 16, DisDP-Colocated's collectives take longer than computation, thus the overall throughput is bounded by collective time. Therefore, DisDP-Colocated's heavier collective traffic causes lower throughput.

3) *Step-Centric Optimizer Pipelining Ablation*: To examine the effect of the step-centric optimizer pipelining, we compare DisDP with DisDP-LC that implements naïve layer-centric pipelining for PS.

Figure 25 shows the throughput of two implementations on the Custom-175B model. We observe that DisDP achieves  $1.10\sim 1.17\times$  higher throughput than DisDP-LC, because DisDP-LC's throughput at large batch sizes is bottlenecked by the layer-centric optimizer that fails to serve line-rate packets. In contrast, DisDP with step-centric pipelining can serve line-rate packets, because the throughput is bottlenecked by the network rather than the optimizer execution.

### F. Scaling Out of DisDP

To show the horizontal scalability of DisDP, we simulate DisDP and ZeRO-Infinity using ASTRA-sim [125], [126] on industrial-scale clusters. We follow the existing industrial practices [47], [64] with 3D parallelism [70], [127] to employ 8-degree TP [128]–[133] over 600GB/s scale-up networks, and 16-degree 1F1B PP [129], [134]–[143] and varying-degree DP over 100Gbps scale-out networks. We vary the DP degree to up to 256, which covers the largest DP degree that industrial practices like Llama-3 [64] (128 DP degree) and MegaScale [47] (192 DP degree) adopts. We run ZeRO-Infinity's collectives on two algorithms: The typical ring algorithms (ZeRO-Inf-Ring), and PAT [144] (ZeRO-Inf-PAT) that performs hierarchical tree-based RS and AG. Figure 26 shows the global TFLOPS with micro-batch sizes of 16 (the same setting in Llama [8]).

We make two observations. First, ZeRO-Infinity with ring algorithm scales poorly with  $>16$  DP degree, because ring collectives introduce a long dependency tree at a large net-

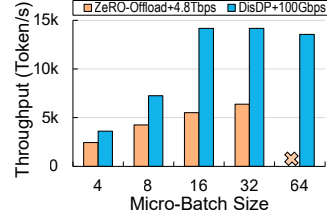


Fig. 27. DisDP under 100Gbps network vs. ZeRO-Offload under 4.8Tbps NVLink.

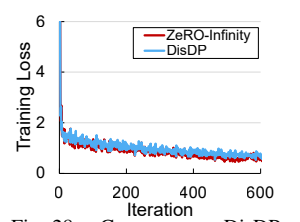


Fig. 28. Convergence: DisDP vs. ZeRO-Infinity

work communication scale [145] and thus is vulnerable to interference [51], [53]. In contrast, DisDP linearly scales out because its hardware-accelerated SmartSwitch-assisted many-to-one collectives compress the dependency chain into a few hops between each worker and the PS. Second, DisDP achieves higher throughput than ZeRO-Infinity with both algorithms. On 256-degree DP, DisDP achieves  $2.0\times$  throughput over ZeRO-Infinity with PAT and  $15.1\times$  throughput over that with the ring algorithm, because ZeRO-Infinity with both algorithms suffers from GEMM-collective interference, while DisDP eliminates the interference by SmartNIC-managed collectives. In conclusion, DisDP's disaggregation of compute, network, and storage enables efficient 3D parallelism with a large DP degree.

### G. Effect of Faster Network

We validate the benefit of DisDP over baselines with higher network bandwidth. Due to a lack of 400Gbps NICs, we choose  $8\times$  A100-40GB GPUs fully connected by 600GB/s NVLink, which is an order of magnitude faster than 100Gb/s of DisDP. We compare DisDP on 100Gbps Ethernet with ZeRO-Offload on a 600GB/s (4.8Tbps) fast network when training the OPT-13B model. Figure 27 shows the results. We make two observations. First, DisDP achieves  $2.22\times$  throughput over NVLink-enhanced ZeRO-Offload, mainly because ZeRO-Offload suffers from severe interference between GEMM and collectives, while DisDP addresses this by disaggregating compute and collectives. Second, DisDP achieves saturated throughput at micro-batch sizes greater than 16 due to GPU compute bottleneck (as demonstrated in Subsection IV-C). Thus, a 100Gbps network rather than high-bandwidth NVLink is sufficient when 1) the micro-batch size is large so that computation takes longer than collectives, and 2) computation and collectives are fully overlapped so collective overhead are completely hidden by computation. In conclusion, we argue for a fully disaggregated design with a relatively slow network, rather than simply upgrading to a faster network under an aggregated infrastructure, to fully address the communication bottleneck in LLM training.

### H. Cost-Efficiency Comparison

To show cost-efficiency benefits of DisDP, we compare the price and throughput of DisDP on a 32-GPU commodity cluster to ZeRO-Infinity on both the 32-GPU commodity cluster and a 32-GPU DGX cluster (intra-machine GPUs are fully connected with 600GB/s NVLink). We use

TABLE VIII  
PRICE OF EACH COMPONENT.

| Component                | Price (\$)   | Component   | Price (\$)   |
|--------------------------|--------------|-------------|--------------|
| Worker w/ CPU and Memory | 9,060 [147]  | SSD         | 850 [148]    |
| PS w/ CPU and Memory     | 10,588 [150] | GPU         | 8,800 [149]  |
| Conventional Switch      | 18,990 [153] | CX-5 NIC    | 755 [151]    |
|                          |              | SmartNIC    | 2,965 [152]  |
|                          |              | SmartSwitch | 10,020 [154] |

TABLE IX  
COST-EFFICIENCY: TRAINING 175B MODEL ON 32 GPUS.

|                         | DisDP on Ours          | ZeRO-Infinity on Ours | ZeRO-Infinity on DGX                   |
|-------------------------|------------------------|-----------------------|----------------------------------------|
| Machine Price (\$)      | 10,600×4<br>+ 17,000×1 | 10,600×4              | 154,800×4<br>(4 machines<br>+ 32 GPUs) |
| GPU Price (\$)          | 8,000×32               | 8,000×32              | + 32 GPUs                              |
| NIC Price (\$)          | 1,600×33               | 1,000×32              | + 32 NICs                              |
| SSD Price (\$)          | 310×12                 | 310×16                | + 16 SSDs)                             |
| Switch Price (\$)       | 10,000×1               | 19,000×1              | 19,000×1                               |
| Total Price (\$)        | 381,920                | 354,360               | 638,200                                |
| Throughput (Token/s)    | 4,386                  | 822                   | 3,745                                  |
| Relative Throughput     | 1.17×                  | 0.22×                 | 1×                                     |
| Relative Price          | 0.60×                  | 0.56×                 | 1×                                     |
| Throughput/Dollar Ratio | 1.96×                  | 0.40×                 | 1×                                     |

throughput-per-dollar as the cost-efficiency metric, so the cost-efficiency benefit is calculated by throughput-per-dollar ratio  $= \frac{\text{Relative Throughput}}{\text{Relative Price}}$ . We gather the price of a DGX machine with 8× A100-40GB GPUs from [146] and the remaining components from wholesalers’ public quotes,<sup>7</sup> as listed in Table VIII. Due to a lack of GPUs, we use ASTRA-sim [125] to simulate the throughput of DisDP and ZeRO-Infinity when training the 175B model. Table IX shows that DisDP only costs 60% of that of a DGX cluster, while achieving 1.17× throughput than ZeRO-Infinity on DGX. As a result, DisDP achieves a competitive 1.96× throughput-per-dollar over ZeRO-Infinity on the DGX cluster mainly due to DisDP’s fully disaggregated design. In contrast, ZeRO-Infinity on the commodity cluster only achieves 0.22× throughput and 0.40× cost-efficiency over that on the DGX cluster.

### I. Training Convergence

To validate that DisDP does not affect the training convergence, we fine-tune the OPT-66B model (the largest pretrained model we can access) on the 8-GPU cluster with the rm-static dataset [155]. During the fine-tuning process, we set the micro-batch size to 8 per worker and the data format to bf16. The fine-tuning process takes ~3 days for ZeRO-Infinity and ~12 hours for DisDP. We compare the training loss of DisDP and ZeRO-Infinity, as shown in Figure 28. We observe that DisDP and ZeRO-Infinity have roughly overlapped loss curves, indicating DisDP keeps the same convergence rate.

## V. RELATED WORK

**SmartNIC-Offloaded Collectives.** Recent works [84]–[86] offload collective scheduling from CPU to SoC-based SmartNICs to overlap network communication and computation. However, they build on the RDMA protocol that only supports reliable one-to-one connections and cannot be applied to

<sup>7</sup>Quote of worker and PS machines is the total price after configuring CPU, memory, system storage, and labor.

many-to-one push/pull. Khalilov et al. [87] enable one-to-many broadcast with SoC-based SmartNIC. However, it does not simultaneously support many-to-one aggregation because an SoC-based SmartNIC cannot process line-rate push and pull simultaneously due to its limited internal switch link and Arm memory bandwidth constraints (Subsection III-B1). In contrast, DisDP uses FPGA-based SmartNICs to enable concurrent many-to-one push and pull for efficient FSDP training.

**Compute-Network Overlapping.** Recent systems propose kernel decomposition [156]–[162] for computation and collectives kernels, so as to enable fine-grained overlapping for TP and expert parallelism where compute and network cannot overlap due to data dependency. However, they do not address the compute-network interference in MSDP, so they still suffer from GPU computing unit contention that prevents compute and network from fully overlapping. In contrast, DisDP enables full compute-network disaggregation with SmartNICs.

Some works enable compute-network overlapping by GPU kernel fusion [163]–[166], which addresses GPU computing unit contention by manually allocating appropriate GPU threads for GEMM and collectives. However, kernel fusion does not address GPU memory bandwidth and L2 contention, so they still suffer from collective bandwidth drop, as shown in Subsection II-A. In contrast, DisDP eliminates the interference with SmartNIC-centric collectives.

**In-Switch Aggregation.** Recent works [66], [67], [114], [167]–[175] leverage SmartSwitches to reduce the network data volume during DNN training. However, these systems only support AllReduce-like semantics, thus not compatible with LLM training. SHARP [122], [123], [176] integrates collective logics on dedicated InfiniBand switches. However, SHARP relies on GPUs for data chunk management and consequently suffers from computation-collective interference. In contrast, DisDP proposes novel smart network-managed collectives to improve GPU utilization.

## VI. CONCLUSION

In this paper, we propose DisDP, a fully disaggregated data-parallel architecture for 100B-scale distributed training. The key idea is 1) offloading collectives to SmartNICs to avoid interference between GEMM kernels and collective kernels, and 2) offloading the optimizer to a scalable PS that supports serving arbitrary numbers of workers. The experimental results show that DisDP achieves 1.96× throughput-per-dollar over ZeRO-Offload on DGX when training on a 175B model.

## ACKNOWLEDGEMENT.

We thank the anonymous ISCA reviewers for improving this paper. The work is supported by the following grants: the Major Project of the Zhejiang Provincial Natural Science Foundation under Grant No. LD26F020002, the National Natural Science Foundation of China under the grant numbers (62472384, 62441236, U24A20326). Zeke Wang is the corresponding author.

## REFERENCES

- [1] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," *arXiv preprint*, 2018.
- [2] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, "Exploring the limits of transfer learning with a unified text-to-text transformer," *J. Mach. Learn. Res.*, 2020.
- [3] M. Zhong, F. Lv, L. Wang, L. Qiu, Y. Wang, Y. Liu, H. Cui, X. Feng, and J. Xue, "Vega: Automatically generating compiler backends using a pre-trained transformer model," in *CGO*, 2025.
- [4] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An image is worth 16x16 words: Transformers for image recognition at scale," *arXiv preprint*, 2020.
- [5] Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, and B. Guo, "Swin transformer: Hierarchical vision transformer using shifted windows," in *ICCV*, 2021.
- [6] M. D. M. Reddy, M. S. M. Basha, M. M. C. Hari, and M. N. Penchalaiah, "Dall-e: Creating images from text," *UGC Care Group I Journal*, 2021.
- [7] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, "Language models are few-shot learners," in *NeurIPS*, 2020.
- [8] LlamaTeam, "The llama 3 herd of models," *arXiv preprint*, 2024.
- [9] S. Smith, M. Patwary, B. Norick, P. LeGresley, S. Rajbhandari, J. Casper, Z. Liu, S. Prabhunoye, G. Zerveas, V. Korthikanti, E. Zhang, R. Child, R. Y. Aminabadi, J. Bernauer, X. Song, M. Shoenybi, Y. He, M. Houston, S. Tiwary, and B. Catanzaro, "Using deepspeed and megatron to train megatron-turing nlq 530b, a large-scale generative language model," *arXiv preprint*, 2022.
- [10] A. Chowdhery, S. Narang, J. Devlin, M. Bosma, G. Mishra, A. Roberts, P. Barham, H. W. Chung, C. Sutton, S. Gehrmann, P. Schuh, K. Shi, S. Tsvyashchenko, J. Maynez, A. Rao, P. Barnes, Y. Tay, N. Shazeer, V. Prabhakaran, E. Reif, N. Du, B. Hutchinson, R. Pope, J. Bradbury, J. Austin, M. Isard, G. Gur-Ari, P. Yin, T. Duke, A. Levskaya, S. Ghemawat, S. Dev, H. Michalewski, X. Garcia, V. Misra, K. Robinson, L. Fedus, D. Zhou, D. Ippolito, D. Luan, H. Lim, B. Zoph, A. Spiridonov, R. Sepassi, D. Dohan, S. Agrawal, M. Omernick, A. M. Dai, T. S. Pillai, M. Pellat, A. Lewkowycz, E. Moreira, R. Child, O. Polozov, K. Lee, Z. Zhou, X. Wang, B. Saeta, M. Diaz, O. Firat, M. Catasta, J. Wei, K. Meier-Hellstern, D. Eck, J. Dean, S. Petrov, and N. Fiedel, "Palm: Scaling language modeling with pathways," *JMLR*, 2023.
- [11] S. Kim, G.-I. Yu, H. Park, S. Cho, E. Jeong, H. Ha, S. Lee, J. S. Jeong, and B.-G. Chun, "Parallax: Sparsity-aware data parallel training of deep neural networks," in *EuroSys*, 2019.
- [12] S. H. Hashemi, S. A. Noghabi, W. Gropp, and R. H. Campbell, "Performance modeling of distributed deep neural networks," *arXiv preprint*, 2016.
- [13] P. Watcharapichat, V. L. Morales, R. C. Fernandez, and P. Pietzuch, "Ako: Decentralised deep learning with partial gradient exchange," in *SoCC*, 2016.
- [14] Y. Li, J. Park, M. Alian, Y. Yuan, Z. Qu, P. Pan, R. Wang, A. Schwing, H. Esmailzadeh, and N. S. Kim, "A network-centric hardware/algorithm co-design to accelerate distributed training of deep neural networks," in *MICRO*, 2018.
- [15] L. Luo, P. West, J. Nelson, A. Krishnamurthy, and L. Ceze, "Plink: Efficient cloud-based training with topology-aware dynamic hierarchical aggregation," in *MLSys*, 2020.
- [16] S. Wang, D. Li, Y. Cheng, J. Geng, Y. Wang, S. Wang, S.-T. Xia, and J. Wu, "Bml: A high-performance, low-cost gradient synchronization algorithm for dml training," in *NIPS*, 2018.
- [17] F. Li, S. Zhao, Y. Qing, X. Chen, X. Guan, S. Wang, G. Zhang, and H. Cui, "Fold3d: Rethinking and parallelizing computational and communicational tasks in the training of large dnn models," *TPDS*, 2023.
- [18] T. Huang, L. Sun, X. Hou, X. Zhu, X. Xia, Y. Wang, M. Chen, and C. Li, "Sub-model parallelism: A scale-out deployment method for large multi-modal dnns," in *CCGrid*, 2024.
- [19] E. Warraich, O. Shabtai, K. Manaa, S. Vargaftik, Y. Piasetzky, M. Kadosh, L. Suresh, and M. Shahbaz, "Ultima: Robust and tail-optimal allreduce for distributed deep learning in the cloud," *arXiv preprint*, 2023.
- [20] Z. Zhu, C. Giannoula, M. Andoorvedu, Q. Su, K. Mangalam, B. Zheng, and G. Pekhimenko, "Mist: Efficient distributed training of large language models via memory-parallelism co-optimization," *arXiv preprint*, 2025.
- [21] Z. Wang, A. Cai, X. Xie, Z. Pan, Y. Guan, W. Chu, J. Wang, S. Li, J. Huang, C. Cai, Y. Hao, and Y. Ding, "Wlb-llm: Workload-balanced 4d parallelism for large language model training," *arXiv preprint*, 2025.
- [22] A. Mohan, R. Walkup, B. Karacali, M. hung Chen, A. Kaya, L. Schour, S. Salaria, S. Wen, I. hsin Chung, A. Alim, C. Evangelinos, L. Luo, M. Dombrowa, L. Schares, A. Sydney, P. Maniotis, S. Koteswara, B. Tang, J. Belog, R. Odaira, V. Tarasov, E. Gampel, D. Thorstensen, T. Gershon, and S. Seelam, "Vela: A virtualized llm training system with gpu direct roce," in *ASPLOS*, 2025.
- [23] Z. Zhu, C. Giannoula, M. Andoorvedu, Q. Su, K. Mangalam, B. Zheng, and G. Pekhimenko, "Mist: Efficient distributed training of large language models via memory-parallelism co-optimization," in *EuroSys*, 2025.
- [24] Z. Jia, L. N. Bhuyan, and D. Wong, "Pccl: Energy-efficient llm training with power-aware collective communication," in *ICCD*, 2024.
- [25] R. Chen, G. Lu, Y. Wang, R. Zhang, Z. Hu, Y. Miao, Z. Cai, J. Leng, and M. Guo, "Baft: bubble-aware fault-tolerant framework for distributed dnn training with hybrid parallelism," *Frontiers of Computer Science*, 2025.
- [26] L. Qin, J. Cui, W. Cai, and J. Huang, "Chimera: Communication fusion for hybrid parallelism in large language models," in *ISCA*, 2025.
- [27] G. Huang, H. Li, L. Qin, J. Huang, Y. Kang, Y. Ding, and Y. Xie, "Traci: Network acceleration of input-dynamic communication for large-scale deep learning recommendation model," in *ISCA*, 2025.
- [28] Z. Zhang, Y. Zhong, Y. Jiang, H. Hu, J. Sun, Z. Ge, Y. Zhu, D. Jiang, and X. Jin, "Distrain: Addressing model and data heterogeneity with disaggregated training for multimodal large language models," in *SIGCOMM*, 2025.
- [29] H. Ge, J. Feng, Q. Huang, F. Fu, X. Nie, L. Zuo, H. Lin, B. Cui, and X. Liu, "Bytescale: Communication-efficient scaling of llm training with a 2048k context length on 16384 gpus," in *SIGCOMM*, 2025.
- [30] L. G. Valiant, "A bridging model for parallel computation," *Communications of the ACM*, 1990.
- [31] P. Patarasuk and X. Yuan, "Bandwidth optimal all-reduce algorithms for clusters of workstations," *Journal of Parallel and Distributed Computing*, 2009.
- [32] T. Chilimbi, Y. Suzue, J. Apacible, and K. Kalyanaraman, "Project adam: Building an efficient and scalable deep learning training system," in *OSDI*, 2014.
- [33] M. Li, D. G. Andersen, J. W. Park, A. J. Smola, A. Ahmed, V. Josifovski, J. Long, E. J. Shekita, and B.-Y. Su, "Scaling distributed machine learning with the parameter server," in *OSDI*, 2014.
- [34] M. Li, D. G. Andersen, A. J. Smola, and K. Yu, "Communication efficient distributed machine learning with the parameter server," in *NIPS*, 2014.
- [35] M. Abadi, P. Barham, J. Chen, Z. Chen, A. Davis, J. Dean, M. Devin, S. Ghemawat, G. Irving, M. Isard, M. Kudlur, J. Levenberg, R. Monga, S. Moore, D. G. Murray, B. Steiner, P. Tucker, V. Vasudevan, P. Warden, M. Wicke, Y. Yu, and X. Zheng, "Tensorflow: a system for large-scale machine learning," in *OSDI*, 2016.
- [36] T. Chen, M. Li, Y. Li, M. Lin, N. Wang, M. Wang, T. Xiao, B. Xu, C. Zhang, and Z. Zhang, "Mxnet: A flexible and efficient machine learning library for heterogeneous distributed systems," *arXiv preprint*, 2015.
- [37] S. Li, Y. Zhao, R. Varma, O. Salpekar, P. Noordhuis, T. Li, A. Paszke, J. Smith, B. Vaughan, P. Damania, and S. Chintala, "Pytorch distributed: Experiences on accelerating data parallel training," in *Vldb*, 2020.
- [38] J. Huang, P. Majumder, S. Kim, A. Muzahid, K. H. Yum, and E. J. Kim, "Communication algorithm-architecture co-design for distributed deep learning," in *ISCA*, 2021.
- [39] L. Luo, J. Nelson, L. Ceze, A. Phanishayee, and A. Krishnamurthy, "Parameter hub: a rack-scale parameter server for distributed deep neural network training," in *SoCC*, 2018.
- [40] A.-L. Jin, W. Xu, S. Guo, B. Hu, and K. Yeung, "Ps+: A simple yet effective framework for fast training on parameter server," *TPDS*, 2022.

- [41] I. Thangakrishnan, D. Cavdar, C. Karakus, P. Ghai, Y. Selivonchik, and C. Pruce, "Herring: Rethinking the parameter server at scale for the cloud," in *SC*, 2020.
- [42] H. Cui, H. Zhang, G. R. Ganger, P. B. Gibbons, and E. P. Xing, "Geeps: Scalable deep learning on distributed gpus with a gpu-specialized parameter server," in *EuroSys*, 2016.
- [43] Y. Jiang, Y. Zhu, C. Lan, B. Yi, Y. Cui, and C. Guo, "A unified architecture for accelerating distributed dnn training in heterogeneous gpu/cpu clusters," in *OSDI*, 2020.
- [44] C. Xie, O. Koyejo, and I. Gupta, "Zenops: A distributed learning system integrating communication efficiency and security," *Algorithms*, 2022.
- [45] Y. Peng, Y. Bao, Y. Chen, C. Wu, and C. Guo, "Optimus: an efficient dynamic resource scheduler for deep learning clusters," in *EuroSys*, 2018.
- [46] W. Wang, M. Khazraee, Z. Zhong, M. Ghobadi, Z. Jia, D. Mudigere, Y. Zhang, and A. Kewitsch, "Topoopt: Co-optimizing network topology and parallelization strategy for distributed training jobs," in *NSDI*, 2023.
- [47] Z. Jiang, H. Lin, Y. Zhong, Q. Huang, Y. Chen, Z. Zhang, Y. Peng, X. Li, C. Xie, S. Nong, Y. Jia, S. He, H. Chen, Z. Bai, Q. Hou, S. Yan, D. Zhou, Y. Sheng, Z. Jiang, H. Xu, H. Wei, Z. Zhang, P. Nie, L. Zou, S. Zhao, L. Xiang, Z. Liu, Z. Li, X. Jia, J. Ye, X. Jin, and X. Liu, "Megascall: Scaling large language model training to more than 10,000 gpus," *arXiv preprint*, 2024.
- [48] J. B. Park, K. Wu, V. S. Maitlthody, Z. Quresh, S. Mahlke, and W.-m. Hwu, "Lsm-gnn: Large-scale storage-based multi-gpu gnn training by optimizing data transfer scheme," *arXiv preprint*, 2024.
- [49] Y. Xu, H. Lee, D. Chen, H. Choi, B. Hechtman, and S. Wang, "Automatic cross-replica sharding of weight update in data-parallel training," *arXiv preprint*, 2020.
- [50] S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He, "Zero: Memory optimizations toward training trillion parameter models," in *SC*, 2020.
- [51] Y. Zhao, A. Gu, R. Varma, L. Luo, C.-C. Huang, M. Xu, L. Wright, H. Shojanazeri, M. Ott, S. Shleifer, A. Desmaison, C. Balioglu, P. Damania, B. Nguyen, G. Chauhan, Y. Hao, A. Mathews, and S. Li, "Pytorch fsdp: Experiences on scaling fully sharded data parallel," in *VLDB*, 2023.
- [52] G. Wang, H. Qin, S. A. Jacobs, C. Holmes, S. Rajbhandari, O. Ruwase, F. Yan, L. Yang, and Y. He, "Zero++: Extremely efficient collective communication for giant model training," in *arXiv preprint*, 2023.
- [53] Z. Zhang, S. Zheng, Y. Wang, J. Chiu, G. Karypis, T. Chilimbi, M. Li, and X. Jin, "Mics: near-linear scaling for training gigantic model on public cloud," in *VLDB*, 2022.
- [54] Q. Chen, Q. Hu, Z. Ye, G. Wang, P. Sun, Y. Wen, and T. Zhang, "Amsp: Super-scaling llm training via advanced model states partitioning," *arXiv preprint*, 2023.
- [55] D. K. Kadiyala, S. Rashidi, T. Heo, A. R. Bambhaniya, T. Krishna, and A. Daglis, "Comet: A comprehensive cluster design methodology for distributed deep learning training," *arXiv preprint*, 2022.
- [56] J. Ren, S. Rajbhandari, R. Y. Aminabadi, O. Ruwase, S. Yang, M. Zhang, D. Li, and Y. He, "Zero-offload: Democratizing billion-scale model training," in *ATC*, 2021.
- [57] S. Rajbhandari, O. Ruwase, J. Rasley, S. Smith, and Y. He, "Zero-infinity: Breaking the gpu memory wall for extreme scale deep learning," in *SC*, 2021.
- [58] J. Fang, Y. Yu, Z. Zhu, S. Li, Y. You, and J. Zhou, "Patrickstar: Parallel training of pre-trained models via chunk-based memory management," *arXiv preprint*, 2021.
- [59] Z. Bian, H. Liu, B. Wang, H. Huang, Y. Li, C. Wang, F. Cui, and Y. You, "Colossal-ai: A unified deep learning system for large-scale parallel training," *arXiv preprint*, 2021.
- [60] Y. Liu, S. Li, J. Fang, Y. Shao, B. Yao, and Y. You, "Colossal-auto: Unified automation of parallelization and activation checkpoint for large-scale models," *arXiv preprint*, 2023.
- [61] J. Ren, J. Luo, K. Wu, M. Zhang, H. Jeon, and D. Li, "Sentinel: Efficient tensor migration and allocation on heterogeneous memory systems for deep learning," in *HPCA*, 2021.
- [62] J. Ren, D. Xu, S. Yang, J. Zhao, Z. Li, C. Navasca, C. Wang, H. Xu, and D. Li, "Enabling large dynamic neural network training with learning-based memory management," in *HPCA*, 2024.
- [63] DeepSeek-AI, "Deepseek-v3 technical report," *arXiv preprint*, 2024.
- [64] W. Chu, X. Xie, J. Yu, J. Wang, A. Phanishayee, C. Tang, Y. Hao, J. Huang, M. Ozdal, J. Wang, V. Goswami, N. Goyal, A. Kadian, A. Gu, C. Cai, F. Tian, X. Wang, M. Si, P. Balaji, C.-H. Chu, and J. Park, "Scaling llama 3 training with efficient parallelism strategies," in *ISCA*, 2025.
- [65] W. Xiong, J. Liu, I. Molybog, H. Zhang, P. Bhargava, R. Hou, L. Martin, R. Rungta, K. A. Sankararaman, B. Oguz, M. Khabasa, H. Fang, Y. Mehdad, S. Narang, K. Malik, A. Fan, S. Bhosale, S. Edunov, M. Lewis, S. Wang, and H. Ma, "Effective long-context scaling of foundation models," in *NAACL-HLT*, 2024.
- [66] C. Lao, Y. Le, K. Mahajan, Y. Chen, W. Wu, A. Akella, and M. Swift, "Atp: In-network aggregation for multi-tenant learning," in *NSDI*, 2021.
- [67] Z. Li, J. Huang, Y. Li, A. Xu, S. Zhou, J. Liu, and J. Wang, "A2tp: Aggregator-aware in-network aggregation for multi-tenant learning," in *EuroSys*, 2023.
- [68] Intel, "Intel tofino programmable ethernet switch asic," <https://www.intel.com/content/www/us/en/products/network-io/programmable-ethernet-switch/tofino-series.html>, 2020.
- [69] P. Bosshart, G. Gibb, H.-S. Kim, G. Varghese, N. McKeown, M. Izard, F. Mujica, and M. Horowitz, "Forwarding metamorphosis: Fast programmable match-action processing in hardware for sdn," *ACM SIGCOMM Computer Communication Review*, 2013.
- [70] C. Zhao, C. Deng, C. Ruan, D. Dai, H. Gao, J. Li, L. Zhang, P. Huang, S. Zhou, S. Ma, W. Liang, Y. He, Y. Wang, Y. Liu, and Y. Wei, "Insights into deepseek-v3: Scaling challenges and reflections on hardware for ai architectures," in *ISCA*, 2025.
- [71] NVIDIA, "Nvidia collective communications library," <https://developer.nvidia.com/nccl>, 2017.
- [72] T. Liu, C. Hei, F. Li, C. Gao, J. Cao, T. Wang, E. Zhai, and X. Wang, "Resccl: Resource-efficient scheduling for collective communication," in *SIGCOMM*, 2025.
- [73] I. S. Olmedo, N. Capodiceci, J. L. Martinez, A. Marongiu, and M. Bertogna, "Dissecting the cuda scheduling hierarchy: a performance and predictability perspective," in *RTAS*, 2020.
- [74] S. Rashidi, M. Denton, S. Sridharan, S. Srinivasan, A. Suresh, J. Nie, and T. Krishna, "Enabling compute-communication overlap in distributed deep learning training platforms," in *ISCA*, 2021.
- [75] NVIDIA, "Multi-process service documentation," <https://docs.nvidia.com/deploy/mps/>, 2017.
- [76] Z. Xiong and N. Zilberman, "Do switches dream of machine learning? toward in-network classification," in *HotNets*, 2019.
- [77] Y. Tokusashi, H. T. Dang, F. Pedone, R. Soulé, and N. Zilberman, "The case for in-network computing on demand," in *EuroSys*, 2019.
- [78] J. Liu, W. Hallahan, C. Schlesinger, M. Sharif, J. Lee, R. Soulé, H. Wang, C. Caşcalav, N. McKeown, and N. Foster, "P4v: Practical verification for programmable data planes," in *SIGCOMM*, 2018.
- [79] Z. Yue, X. Xiang, F. Tu, Y. Wang, Y. Wang, S. Wei, Y. Hu, and S. Yin, "15.1 a 0.795 fJ/bit physically-unclonable function-protected team for a software-defined networking switch," in *ISSCC*, 2024.
- [80] X. Gao, J. Gao, K. K. G. M. Haseeb, E. Zhai, B. Dong, J. Tassarotti, S. Narayana, and A. Sivaraman, "Parserhawk: Hardware-aware parser generator using program synthesis," in *SIGCOMM*, 2025.
- [81] NVIDIA, "Using nvidia sharp with nvidia nccl," <https://docs.nvidia.com/networking/display/sharpv300/using+nvidia+sharp+with+nccl>.
- [82] W. An, X. Bi, G. Chen, S. Chen, C. Deng, H. Ding, K. Dong, Q. Du, W. Gao, K. Guan, J. Guo, Y. Guo, Z. Fu, Y. He, P. Huang, J. Li, W. Liang, X. Liu, X. Liu, Y. Liu, Y. Liu, S. Lu, X. Lu, X. Nie, T. Pei, J. Qiu, H. Qu, Z. Ren, Z. Sha, X. Su, X. Sun, Y. Tan, M. Tang, S. Wang, Y. Wang, Y. Wang, Z. Xie, Y. Xiong, Y. Xu, S. Ye, S. Yu, Y. Zha, L. Zhang, H. Zhang, M. Zhang, W. Zhang, Y. Zhang, C. Zhao, Y. Zhao, S. Zhou, S. Zhou, and Y. Zou, "Fire-flyer ai-hpc: A cost-effective software-hardware co-design for deep learning," in *SC*, 2024.
- [83] NVIDIA, "Nvidia bluefield platform," <https://www.nvidia.com/en-us/networking/products/data-processing-unit/> (accessed Feb. 21, 2026).
- [84] K. K. Suresh, B. Michalowicz, B. Ramesh, N. Contini, J. Yao, S. Xu, A. Shafi, H. Subramoni, and D. Panda, "A novel framework for efficient offloading of communication operations to bluefield smartnics," in *IPDPS*, 2023.
- [85] R. Graham, G. Bosilca, Y. Qin, B. Settlemeyer, G. Shainer, C. Stunkel, G. Vallee, B. Williams, G. Cisneros-Stoianowski, S. Ohlmann, and M. Rampp, "Optimizing application performance with bluefield: accelerating large-message blocking and nonblocking collective operations," in *ISC*, 2024.
- [86] M. Usman, M. Benito, S. Iserte, and A. J. Peña, "Odos-mpi: Hpc-friendly smartnic offloading of computation/communication kernels," in *SC*, 2025.

- [87] M. Khalilov, S. Di Girolamo, M. Chrapek, R. Nudelman, G. Bloch, and T. Hoefler, "Network-offloaded bandwidth-optimal broadcast and allgather for distributed ai," in *SC*, 2024.
- [88] P. Fleming, C. Chang, D. Collier, A. Singhai, S. Doyle, E. Louzoun, D. Lee, V. Ayyavu, S. Livne, R. Hathaway, T. Hurson, J. Ellis, T. Bar-Kanarik, J. Kenny, C. Dumitrescu, and Y. Wolberger, "Intel® ipu e2200: Second generation infrastructure processing unit (ipu)," in *HCS*, 2025.
- [89] X. Chen, J. Zhang, B. Zhu, X. Zhu, Z. Chen, T. Fu, S. Ma, L. Zhu, C. Shi, Y. Zhang, Y. Shu, P. Cheng, and Z. Wang, "Flexins: A smartnic-centric, line-rate and flexible network stack," in *EuroSys*, 2026.
- [90] NVIDIA, "Nvidia bluefield-2 infiniband/ethernet dpu specifications," <https://docs.nvidia.com/networking/display/bluefield2dpuvpi/specifications> (accessed Mar. 6, 2026).
- [91] —, "Nvidia bluefield-3 networking platform specifications," <https://docs.nvidia.com/networking/display/bf3dpu/specifications> (accessed Feb. 21, 2026).
- [92] Y. Yuan, J. Huang, Y. Sun, T. Wang, J. Nelson, D. R. Ports, Y. Wang, R. Wang, C. Tai, and N. S. Kim, "Rambda: Rdma-driven acceleration framework for memory-intensive  $\mu$ s-scale datacenter applications," in *HPCA*, 2023.
- [93] J. Zhang, H. Huang, L. Zhu, S. Ma, D. Rong, Y. Hou, M. Sun, C. Gu, P. Cheng, C. Shi, and Z. Wang, "Smartds: Middle-tier-centric smartnic enabling application-aware message split for disaggregated block storage," in *ISCA*, 2023.
- [94] J. Cong, Z. Fang, M. Lo, H. Wang, J. Xu, and S. Zhang, "Understanding performance differences of fpgas and gpus," in *FCCM*, 2018.
- [95] A. Caulfield, P. Costa, and M. Ghobadi, "Beyond smartnics: Towards a fully programmable cloud," in *HPSR*, 2018.
- [96] D. Firestone, A. Putnam, S. Mundkur, D. Chiou, A. Dabagh, M. Andrewartha, H. Angepat, V. Bhanu, A. Caulfield, E. Chung, H. K. Chandrappa, S. Chaturmohta, M. Humphrey, J. Lavier, N. Lam, F. Liu, K. Ovtcharov, J. Padhye, G. Popuri, S. Raindel, T. Sapre, M. Shaw, G. Silva, M. Sivakumar, N. Srivastava, A. Verma, Q. Zuhair, D. Bansal, D. Burger, K. Vaid, D. A. Maltz, and A. Greenberg, "Azure accelerated networking: Smartnics in the public cloud," in *NSDI*, 2018.
- [97] B. Li, Z. Ruan, W. Xiao, Y. Lu, Y. Xiong, A. Putnam, E. Chen, and L. Zhang, "Kv-direct: High-performance in-memory key-value store with programmable nic," in *SOSP*, 2017.
- [98] S. Choi, M. Shahbaz, B. Prabhakar, and M. Rosenblum, "λ-nic: Interactive serverless compute on programmable smartnics," in *ICDCS*, 2020.
- [99] A. Guo, T. Geng, Y. Zhang, P. Haghi, C. Wu, C. Tan, Y. Lin, A. Li, and M. Herbordt, "A framework for neural network inference on fpga-centric smartnics," in *FPL*, 2022.
- [100] H. Ji, M. Mansi, Y. Sun, Y. Yuan, J. Huang, R. Kuper, M. M. Swift, and N. S. Kim, "Styx: Exploiting smartnic capability to reduce datacenter memory tax," in *ATC*, 2023.
- [101] Z. Guo, J. Lin, Y. Bai, D. Kim, M. Swift, A. Akella, and M. Liu, "Lognic: A high-level performance model for smartnics," in *MICRO*, 2023.
- [102] J. Lin, Z. Guo, M. Shah, T. Ji, Y. Zhang, D. Kim, and A. Akella, "Enabling portable and high-performance smartnic programs with alkali," in *NSDI*, 2025.
- [103] J. Lu, S. Zhu, J. Liang, Y. Lin, T. Pan, Y. Qiao, Y. Song, W. Su, Y. Xie, Y. Li, E. Song, S. Zhang, X. Sun, R. Wen, X. Wei, B. Lyu, and X. Li, "Albatross: A containerized cloud gateway platform with fpga-accelerated packet-level load balancing," in *SIGCOMM*, 2025.
- [104] X. Li, E. Song, B. Yang, T. Pan, Y. Yang, Q. Fu, Y. Song, Y. Lv, Z. Chen, J. Lu, S. Zhang, X. Sun, R. Wen, X. Wei, B. Lyu, Z. Zong, Q. He, and S. Zhu, "Nezha: Smartnic-based virtual switch load sharing," in *SIGCOMM*, 2025.
- [105] D. Cock, A. Ramdas, D. Schwyn, M. Giardino, A. Turowski, Z. He, N. Hossle, D. Korolija, M. Licciardello, K. Martsenko, R. Achermann, G. Alonso, and T. Roscoe, "Enzian: an open, general, cpu/fpga platform for systems software research," in *ASPLOS*, 2022.
- [106] D. Korolija, T. Roscoe, and G. Alonso, "Do OS abstractions make sense on FPGAs?" in *OSDI*, 2020.
- [107] J. Lin, K. Patel, B. E. Stephens, A. Sivaraman, and A. Akella, "Panic: A high-performance programmable nic for multi-tenant networks," in *OSDI*, 2020.
- [108] A. Guo, T. Geng, Y. Zhang, P. Haghi, C. Wu, C. Tan, Y. Lin, A. Li, and M. Herbordt, "Fcsn: A fpga-centric smartnic framework for neural networks," in *FCCM*, 2022.
- [109] Y. Li, J. Lou, S. Vanavasam, and N. S. Kim, "Hint: A hardware platform for intra-host nic traffic and smartnic emulation," *IEEE Computer Architecture Letters*, 2025.
- [110] Z. He, D. Korolija, Y. Zhu, B. Ramhorst, T. Laan, L. Petrica, M. Blott, and G. Alonso, "Accl+: an fpga-based collective engine for distributed applications," in *OSDI*, 2024.
- [111] Z. Wang, H. Huang, J. Zhang, F. Wu, and G. Alonso, "Fpganic: An fpga-based versatile 100gb smartnic for gpus," in *ATC*, 2022.
- [112] D. Sidler, Z. Wang, M. Chiosa, A. Kulkarni, and G. Alonso, "Strom: Smart remote memory," in *EuroSys*, 2020.
- [113] P. Micikevicius, S. Narang, J. Alben, G. Damos, E. Elsen, D. Garcia, B. Ginsburg, M. Houston, O. Kuchaiev, G. Venkatesh, and H. Wu, "Mixed precision training," *arXiv preprint*, 2017.
- [114] A. Sapio, M. Canini, C.-Y. Ho, J. Nelson, P. Kalnis, C. Kim, A. Krishnamurthy, M. Moshref, D. Ports, and P. Richtárik, "Scaling distributed machine learning with in-network aggregation," in *NSDI*, 2021.
- [115] S. Zhang, S. Roller, N. Goyal, M. Artetxe, M. Chen, S. Chen, C. Dewan, M. Diab, X. Li, X. V. Lin, T. Mihaylov, M. Ott, S. Shleifer, K. Shuster, D. Simig, P. S. Koura, A. Sridhar, T. Wang, and L. Zettlemoyer, "Opt: Open pre-trained transformer language models," *arXiv preprint*, 2022.
- [116] Edgecore, "Edgecore wedge100bf-32x product info," <https://www.edge-core.com/productsInfo.php?cls=1&cls2=5&cls3=181&id=335>, 2020.
- [117] T. Chen, B. Xu, C. Zhang, and C. Guestrin, "Training deep nets with sublinear memory cost," *arXiv preprint*, 2016.
- [118] P. Jain, A. Jain, A. Nrusimha, A. Gholami, P. Abbeel, J. Gonzalez, K. Keutzer, and I. Stoica, "Checkmate: Breaking the memory wall with optimal tensor rematerialization," *MLSys*, 2020.
- [119] NVIDIA, "Sn2700 open ethernet switch," <https://network.nvidia.com/files/doc-2020/pb-sn2700.pdf>, 2022.
- [120] J. Rasley, S. Rajbhandari, O. Ruwase, and Y. He, "Deepspeed: System optimizations enable training deep learning models with over 100 billion parameters," in *SIGKDD*, 2020.
- [121] A. Paszke, S. Gross, S. Chintala, G. Chanan, E. Yang, Z. DeVito, Z. Lin, A. Desmaison, L. Antiga, and A. Lerer, "Automatic differentiation in pytorch," in *Autodiff@NIPS*, 2017.
- [122] R. L. Graham, D. Bureddy, P. Lui, H. Rosenstock, G. Shainer, G. Bloch, D. Goldenberg, M. Dubman, S. Kotchubievsky, V. Koushnir, L. Levi, A. Margolin, T. Ronen, A. Shpiner, O. Wertheim, and E. Zahavi, "Scalable hierarchical aggregation protocol (sharp): A hardware architecture for efficient data reduction," in *COMHPC@SC*. IEEE, 2016, pp. 1–10.
- [123] R. L. Graham, L. Levi, D. Burreddy, G. Bloch, D. C. Gilad Shainer, G. Elias, D. Klein, J. Ladd, O. Maor, V. P. Ami Marelli, E. Romlet, Y. Qin, and I. Zemah, "Scalable hierarchical aggregation and reduction protocol (sharp) streaming-aggregation hardware design and evaluation," in *ISC High Performance*, 2020.
- [124] NVIDIA, "Qm8790 mellanox quantum™ hdr edge switch," <https://network.nvidia.com/files/doc-2020/pb-qm8790.pdf>, 2022.
- [125] W. Won, T. Heo, S. Rashidi, S. Sridharan, S. Srinivasan, and T. Krishna, "Astra-sim2.0: Modeling hierarchical networks and disaggregated systems for large-model training at scale," in *ISPASS*, 2023.
- [126] S. Sridharan, T. Heo, L. Feng, Z. Wang, M. Bergeron, W. Fu, S. Zheng, B. Coutinho, S. Rashidi, C. Man, and T. Krishna, "Chakra: Advancing performance benchmarking and co-design using standardized execution traces," in *Benchmarking Machine Learning Workloads on Emerging Hardware@MLSys*, 2023.
- [127] S. Rashidi, W. Won, S. Srinivasan, P. Gupta, and T. Krishna, "Fred: A wafer-scale fabric for 3d parallel dnn training," in *ISCA*, 2025.
- [128] M. Shoenybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, "Megatron-lm: Training multi-billion parameter language models using model parallelism," *arXiv preprint*, 2019.
- [129] D. Narayanan, M. Shoenybi, J. Casper, P. LeGresley, M. Patwary, V. Korthikanti, D. Vainbrand, P. Kashinkunti, J. Bernauer, B. Catanzaro, and A. Phanishayee, "Efficient large-scale language model training on gpu clusters using megatron-lm," in *SC*, 2021.
- [130] Q. Xu, S. Li, C. Gong, and Y. You, "An efficient 2d method for training super-large deep learning models," *arXiv preprint*, 2021.
- [131] Z. Bian, Q. Xu, B. Wang, and Y. You, "Maximizing parallelism in distributed training for huge neural networks," *arXiv preprint*, 2021.
- [132] A. Agrawal, A. Panwar, J. Mohan, N. Kwatra, B. S. Gulavani, and R. Ramjee, "Sarathi: Efficient llm inference by piggybacking decodes with chunked prefills," *arXiv preprint*, 2023.

- [133] H. Nam, G. Gerogiannis, and J. Torrellas, “Meshslice: Efficient 2d tensor parallelism for distributed dnn training,” in *ISCA*, 2025.
- [134] Y. Huang, Y. Cheng, A. Bapna, O. Firat, D. Chen, M. Chen, H. Lee, J. Ngiam, Q. V. Le, Y. Wu, and zhifeng Chen, “Gpipe: Efficient training of giant neural networks using pipeline parallelism,” in *NeurIPS*, 2019.
- [135] D. Narayanan, A. Phanishayee, K. Shi, and X. Chen, “Memory-efficient pipeline-parallel dnn training,” in *ICML*, 2021.
- [136] S. Fan, Y. Rong, C. Meng, Z. Cao, S. Wang, Z. Zheng, C. Wu, G. Long, J. Yang, L. Xia, L. Diao, X. Liu, and W. Li, “Dapple: A pipelined data parallel approach for training large models,” in *PPoPP*, 2021.
- [137] Y. Feng, M. Xie, Z. Tian, S. Wang, Y. Lu, and J. Shu, “Mobius: Fine tuning large-scale models on commodity gpu servers,” in *ASPLOS*, 2023.
- [138] P. Qi, X. Wan, G. Huang, and M. Lin, “Zero bubble pipeline parallelism,” *arXiv preprint*, 2023.
- [139] D. Narayanan, A. Phanishayee, K. Shi, X. Chen, and M. Zaharia, “Memory-efficient pipeline-parallel dnn training,” in *ICML*, 2021.
- [140] Z. Sun, H. Cao, Y. Wang, G. Feng, S. Chen, H. Wang, and W. Chen, “Adapipeline: Optimizing pipeline parallelism with adaptive recomputation and partitioning,” in *ASPLOS*, 2024.
- [141] S. Zhao, F. Li, X. Chen, X. Guan, J. Jiang, D. Huang, Y. Qing, S. Wang, P. Wang, G. Zhang, C. Li, P. Luo, and H. Cui, “vpipeline: A virtualized acceleration system for achieving efficient and scalable pipeline parallel dnn training,” *TPDS*, 2021.
- [142] J. H. Park, G. Yun, M. Y. Chang, N. T. Nguyen, S. Lee, J. Choi, S. H. Noh, and Y.-r. Choi, “Hetpipe: Enabling large dnn training on (whimpy) heterogeneous gpu clusters through integration of pipelined model parallelism and data parallelism,” in *ATC*, 2020.
- [143] Z. Qu, D. Niu, S. Li, H. Zheng, and Y. Xie, “Tt-gnn: Efficient on-chip graph neural network training via embedding reformation and hardware optimization,” in *MICRO*, 2023.
- [144] S. Jeagey, “Pat: a new algorithm for all-gather and reduce-scatter operations at scale,” *arXiv preprint*, 2025.
- [145] Z. Chen, X. Liu, M. Li, Y. Hu, H. Mei, H. Xing, H. Wang, W. Shi, S. Liu, and Y. Xu, “Rina: Enhancing ring-allreduce with in-network aggregation in distributed model training,” in *ICNP*, 2024.
- [146] Router-switch, “Dell dgx a100 price list,” <https://itprice.com/dell-price-list/a100%20p3687%20320gb.html> (accessed Nov. 13, 2025).
- [147] Acme Micro Systems, “Supermicro superserver 4029gp-trt 4u dual socket,” [https://www.acmemicro.com/Product/16479/Supermicro-SuperServer-4029GP-TRT-4U-Dual-socket-P-\(LGA-3647\)-24xDDR4-8xGPU-2x10GbE-11xPCIe-R2000W-SYS-4029GP-TRT](https://www.acmemicro.com/Product/16479/Supermicro-SuperServer-4029GP-TRT-4U-Dual-socket-P-(LGA-3647)-24xDDR4-8xGPU-2x10GbE-11xPCIe-R2000W-SYS-4029GP-TRT) (accessed Nov. 13, 2025).
- [148] —, “Intel ssdpf2kx038t11z - 3.84tb ssd nvme u.2 15mm,” <https://www.acmemicro.com/Product/18775/Intel-SSDPF2KX038T11Z---3-84TB-SSD-NVMe-U-2-15mm-PCIe-4-0-D7-P5520-Form-6500-MB-s-Read-3D4-TLC-NAND> (accessed Nov. 13, 2025).
- [149] Network Outlet, “Nvidia a100 40gb pcie gpu,” <https://networkoutlet.com/products/nvidia-a100-40gb-pcie-gpu-ampere-architecture-with-nvlink-mig> (accessed Nov. 13, 2025).
- [150] Acme Micro Systems, “Superworksta sys-740a-t tower xeon dual socket p+,” [https://www.acmemicro.com/Product/18368/SuperWorksta-SYS-740A-T-Tower-Xeon-Dual-Socket-P+-\(LGA-4189\)-DDR4-8x-3-5-SATA-2x-1GbE-6-PCI-E-Dual-1200W](https://www.acmemicro.com/Product/18368/SuperWorksta-SYS-740A-T-Tower-Xeon-Dual-Socket-P+-(LGA-4189)-DDR4-8x-3-5-SATA-2x-1GbE-6-PCI-E-Dual-1200W) (accessed Nov. 13, 2025).
- [151] —, “Mellanox mcx515a-ccat connectx-5 en network interface card 100gbe single-port qsfp28 pcie3.0 x16 tall bracket,” <https://www.acmemicro.com/Product/16314/Mellanox-MCX515A-CCAT-ConnectX-5-EN-Network-Interface-Card-100GbE-Single-Port-QSFP28-PCIe3-0-x16-Tall-Bracket> (accessed Nov. 13, 2025).
- [152] AMD, “Amd alveo u50 data center accelerator card,” <https://www.amd.com/en/products/accelerators/alveo/u50/a-u50-p00g-pq-g.html> (accessed Nov. 13, 2025).
- [153] Acme Micro Systems, “Mellanox msn2700-cs2f 100gbe, 1u open ethernet switch w/mlnx-os,32 qsfp28 ports,” [https://www.acmemicro.com/Product/15874/Mellanox-MSN2700-CS2F-100GbE-1U-Open-Ethernet-Switch-w-MLNX-OS-32-QSFP28-ports-2-PS\(AC\)-x86-CPU-Standarddepth-P2C-airflow](https://www.acmemicro.com/Product/15874/Mellanox-MSN2700-CS2F-100GbE-1U-Open-Ethernet-Switch-w-MLNX-OS-32-QSFP28-ports-2-PS(AC)-x86-CPU-Standarddepth-P2C-airflow) (accessed Nov. 13, 2025).
- [154] Colfax Direct, “Edgecore wedge 100bf-32x 32-port 100gbe bare metal switch with onie,” <https://www.colfaxdirect.com/store/pc/viewPrd.asp?idproduct=3485> (accessed Nov. 13, 2025).
- [155] “rm-static dataset at huggingface,” <https://huggingface.co/datasets/Dahoas/rm-static>.
- [156] S. Wang, J. Wei, A. Sabne, A. Davis, B. Ilbeyi, B. Hechtman, D. Chen, K. S. Murthy, M. Maggioni, Q. Zhang, S. Kumar, T. Guo, Y. Xu, and Z. Zhou, “Overlap communication with dependent computation via decomposition in large deep learning models,” in *ASPLOS*, 2022.
- [157] C. Chen, X. Li, Q. Zhu, J. Duan, P. Sun, X. Zhang, and C. Yang, “Centauri: Enabling efficient scheduling for communication-computation overlap in large model training via communication partitioning,” in *ASPLOS*, 2024.
- [158] C. Jiang, Y. Tian, Z. Jia, C. Wu, Y. Wang, and S. Zheng, “Lancet: Accelerating mixture-of-experts training by overlapping weight gradient computation and all-to-all communication,” in *MLSys*, 2024.
- [159] K. Hong, X. Li, M. Liu, Q. Mao, T. Wu, Z. Huang, L. Chen, Z. Wang, Y. Zhang, Z. Zhu, G. Dai, and Y. Wang, “Efficient and adaptable overlapping for computation and communication via signaling and reordering,” in *EuroSys*, 2026.
- [160] H. Huang, Y. Li, J. Sun, X. Zhu, J. Zhang, L. Luo, J. Li, and Z. Wang, “P4sgd: Programmable switch enhanced model-parallel training on generalized linear models on distributed fpgas,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 34, no. 8, pp. 2311–2324, 2023.
- [161] S. A. Fahmy, Z. Yang, Y. Chen, G. Alonso, Z. István, and M. Canini, “Fpgas are the hero in-network computing needs,” in *APSys*, 2025.
- [162] M. Venere, G. Sorrentino, B. Ramhorst, M. J. Heer, L. Petrica, D. Korolija, M. D. Santambrogio, D. Conficconi, G. Alonso, and K. O’Brien, “Ropeerto: A datacenter-scale architecture for peer-to-peer dma between gpus and fpgas,” in *EuroSys*, 2026.
- [163] A. Jangda, J. Huang, G. Liu, A. H. N. Sabet, S. Maleki, Y. Miao, M. Musuvathi, T. Mytkowicz, and O. Saarikivi, “Breaking the computation and communication abstraction barrier in distributed machine learning workloads,” in *ASPLOS*, 2022.
- [164] S. Zheng, J. Fang, X. Zheng, Q. Hou, W. Bao, N. Zheng, Z. Jiang, D. Wang, J. Ye, H. Lin, L.-W. Chang, and X. Liu, “Tilelink: Generating efficient compute-communication overlapping kernels using tile-centric primitives,” in *MLSys*, 2025.
- [165] K. Punniyamurthy, K. Hamidouche, and B. M. Beckmann, “Optimizing distributed ml communication with fused computation-collective operations,” in *SC*, 2024.
- [166] X. Cheng, Z. Zhang, Y. Zhou, J. Ji, J. Jiang, Z. Zhao, Z. Xiao, Z. Ye, Y. Huang, R. Lai, H. Jin, B. Hou, M. Wu, Y. Dong, A. Yip, Z. Ye, S. Wang, W. Yang, X. Miao, T. Chen, and Z. Jia, “Mirage persistent kernel: A compiler and runtime for mega-kernelizing tensor programs,” *arXiv preprint*, 2025.
- [167] L. Mai, L. Ruppert, A. Alim, P. Costa, M. Migliavacca, P. Pietzuch, and A. L. Wolf, “Netagg: Using middleboxes for application-specific on-path aggregation in data centres,” in *CoNEXT*, 2014.
- [168] M. Liu, L. Luo, J. Nelson, L. Ceze, A. Krishnamurthy, and K. Atreya, “Incbriks: Toward in-network computation with an in-network cache,” in *ASPLOS*, 2017.
- [169] H. Wang, Y. Qin, C. Lao, Y. Le, W. Wu, and K. Chen, “Efficient data-plane memory scheduling for in-network aggregation,” *arXiv preprint*, 2022.
- [170] M. Scazzariello, T. Caiazz, H. Ghasemirahni, T. Barbette, D. Kostić, and M. Chiesa, “A high-speed stateful packet processing approach for tbps programmable switches,” in *NSDI*, 2023.
- [171] Y. Li, I.-J. Liu, Y. Yuan, D. Chen, A. Schwing, and J. Huang, “Accelerating distributed reinforcement learning with in-switch computing,” in *ISCA*, 2019.
- [172] B. Reidys, Y. Xue, D. Li, B. Sukhwani, W.-M. Hwu, D. Chen, S. Asaad, and J. Huang, “Rackblox: A software-defined rack-scale storage system with network-storage co-design,” in *SOSP*, 2023.
- [173] T. Swamy, A. Rucker, M. Shahbaz, I. Gaur, and K. Olukotun, “Taurus: a data plane architecture for per-packet ml,” in *ASPLOS*, 2022.
- [174] B. Klenk, N. Jiang, G. Thorson, and L. Dennison, “An in-network architecture for accelerating shared-memory multiprocessor collectives,” in *ISCA*, 2020.
- [175] S. Liu, Q. Wang, J. Zhang, W. Wu, Q. Lin, Y. Liu, M. Xu, M. Canini, R. C. Cheung, and J. He, “In-network aggregation with transport transparency for distributed training,” in *ASPLOS*, 2023.
- [176] S. Dong, Z. Niu, M. Zhang, Z. Xu, C. Hu, W. Wang, P. Zhu, Q. Song, L. Qu, P. Cheng, Y. Xiong, C. Tian, C. Nguyen, and X. Wang, “Mina: Auto-scale in-network aggregation for machine learning service,” in *APNet*, 2023.